



University of Pisa

Department of Information Engineering

Master's Degree in Electronic Engineering
Microelectronic System Design

Design of a function generator using CycloneII FPGA

Academic Year 2025/2026

Students:

Dario Aliberti
Tommaso Tonarini

Contents

1	Introduction	2
1.1	DDFS technique	2
1.1.1	Amplitude quantization	3
1.1.2	Frequency quantization	3
1.1.3	LUT optimization	4
1.2	Parallel bus DAC	5
1.2.1	R-2R ladder DAC	5
1.3	Button matrix keypad	6
1.4	LCD communication	7
1.4.1	Character display	7
1.4.2	Timing Characteristics	8
2	Description of the architecture	9
2.1	Specifications	9
2.2	Module description	10
2.2.1	DDFS module	10
2.2.2	Keypad frequency select module	17
2.3	LCD module	25
2.4	Clock distribution	32
3	Test-plan	33
3.1	Functional verification	33
3.1.1	DDFS module	33
3.1.2	Keypad frequency select module	36
3.1.3	LCD module	40
3.2	On-board module testing	43
3.2.1	DDFS module	43
3.2.2	Keypad frequency select module	44
3.2.3	LCD module	45
3.2.4	Physical test of the entire design	46
4	Implementation on Quartus	50
4.1	Warnings	50
4.1.1	LCD module	50
4.1.2	DDFS module	51
4.1.3	Keypad frequency module	51
4.1.4	Other warnings	52
4.2	Timing analyzer	54
4.2.1	Timing configuration file	54
4.2.2	Timing slack and critical path	57
4.3	Power consumption	59
4.4	Resource usage	60
5	Conclusions	62
5.1	Criticality	62
5.2	Possible optimizations	63
5.3	Possible upgrades	64

1 Introduction

Frequency generators are electronic devices widely used in laboratories to generate electrical waves with the desired shapes. The most common waveforms used are sinusoidal, triangular, and square, but sometimes sinc, noise, and other less common waves are also used. To generate an analog waveform with a digital circuit, different techniques are used, but each one is made of these blocks

- **Human-Machine interface**

It is used to choose the waveform to be generated

- **Digital waveform generator**

It is the digital block that sends data to be converted into analog signals with the right timing specifications

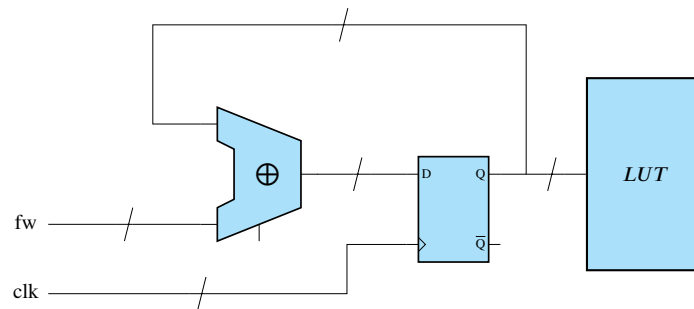
- **Digital analog converter**

It is the component that converts the digital signals into analog signals

The main blocks used to implement the function generator will now be briefly described.

1.1 DDFS technique

The DDFS technique also called **Direct Digital Frequency synthesis** is a type of digital electronic signal generator that read the data stored in a memory called LUT or **Look Up Table** to generate the samples for the desired signal.



The DDFS signal generator is based on a counter called **phase accumulator**.

The output of this counter is increased, at every positive edge of the clock signal, by the value stored in the variable f_w , called **frequency word**.

Then, the output of the counter is used as the address for a Look-Up Table memory that contains the samples of the signal to be generated.

1.1.1 Amplitude quantization

The digital output value from the LUT is sent to an external **digital to analog converter**, and the number of bits used to store each sample of the wave determines the smallest amplitude swing that the wave can produce.

If we use a **C2 balanced representation** to generate the wave, the smallest output voltage swing is

$$\min(\Delta V_{out}) = \frac{V_{cc}}{2^N - 1}$$

where N is the number of bits for the signal samples and V_{cc} is the maximum output voltage of the DAC.

1.1.2 Frequency quantization

The samples are sent to the DAC at the clock frequency; indeed, at every positive edge of the clock, the flip-flop updates the output and the adder increases the input of the register for the next address value.

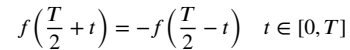
The address does not increase by a unit amount, but depends on the value of f_w ; with this procedure, we can control the frequency of the output wave simply by adjusting the value of f_w .

The output frequency of the wave is

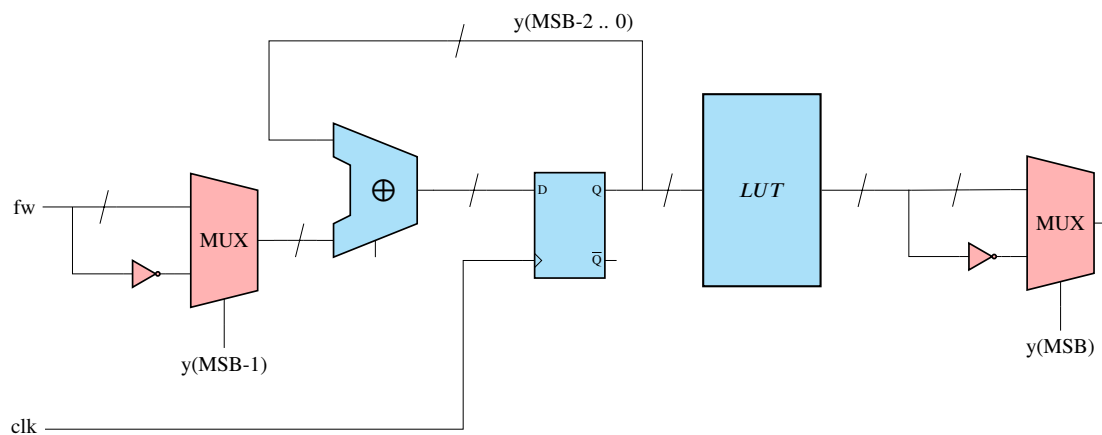
$$f = f_{clk} \cdot \frac{f_w}{LUT_DIM}$$

where f_{clk} is the clock frequency of the DDFS and LUT_DIM is the number of samples stored in the LUT. The problem is that increasing the value of f_w involves an increase in the output wave frequency, but a reduction in the number of samples used for the generation of the wave, which results in a stepped output signal.

Some types of waves have a quadrangular-symmetrical profile, meaning a periodic signal that, every quarter of its period, is reproduced by a vertical symmetry, and every half of its period by a horizontal symmetry.



Vertical symmetry should be applied during the second and fourth quarters of the period, while horizontal symmetry should be applied during the second half of the period. Therefore, if the LUT size is a power of two, the vertical symmetry signal can be $y(MSB - 1)$ and the horizontal symmetry signal $y(MSB)$, where $y(n)$ is the n -th output bit of the phase accumulator. Using a C2 representation the **LUT optimization circuit** can be done as follows



1.2 Parallel bus DAC

The DAC, or **digital to analog converter**, is a device that takes a digital value as input and provides an analog output signal proportional to it.

The output voltage of the DAC can be calculated with the following formula

$$V_{out} = \frac{n}{2^N - 1} \cdot V_{max}$$

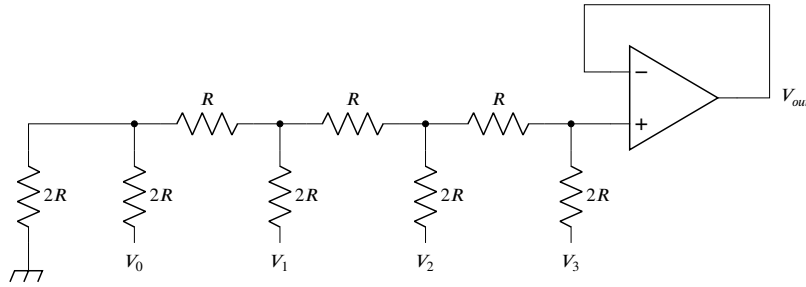
where N is the number of digital input bits, n is the input value of the DAC, and V_{max} is the maximum output voltage of the DAC.

For the most common types of DACs, the input signal is expressed as an unsigned value represented in C1 notation.

1.2.1 R-2R ladder DAC

One of the most common types of DAC is the R-2R ladder, where the output voltage depends on the input voltage values $V_0 \dots V_N$.

The output of the ladder is usually buffered with an operational amplifier to ensure that the output resistance seen from the ladder is always high, independently of the value of the load resistance.



The output voltage for this type of DAC can be calculated as follows

$$V_{out}[4 \text{ bit}] = \frac{2^0 \cdot V_0 + 2^1 \cdot V_1 + 2^2 \cdot V_2 + 2^3 \cdot V_3}{2^4} \Rightarrow V_{out}[n \text{ bit}] = \frac{1}{2^N} \cdot \sum_{i=0}^{N-1} (2^i \cdot V_i)$$

where the first formula refers to a 4-bit R-2R ladder DAC (as shown in the image above), and the second formula refers to a generic N-bit DAC.

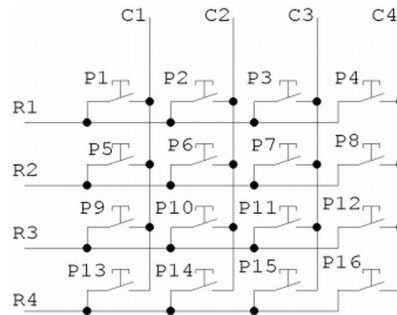
The most critical aspect of using this type of DAC is that the output voltage error is proportional to the mismatch of the resistor values and that the voltage across the resistors dissipates power.

For this reasons, they are not recommended for low-power or high-precision applications.

1.3 Button matrix keypad

The **button matrix keypad** is a device used to interface an electrical device with the user.

The device is composed of multiple buttons, and usually, to reduce the number of connection pins, the buttons are connected in a matrix, as shown in the figure below.



To read the button pressed the rows must be connected to $0V$ or V_{cc} and the column have to be read following this sequence:

- $R1 = V_{cc}$ $R2 = 0V$ $R3 = 0V$ $R4 = 0V$
the first row is enabled and the others are disabled, so the $C_1 \dots C_4$ read values are referred to $P1 \dots P4$
- $R1 = 0V$ $R2 = V_{cc}$ $R3 = 0V$ $R4 = 0V$
the second row is enabled and the others are disabled, so the $C_1 \dots C_4$ read values are referred to $P5 \dots P8$
- $R1 = 0V$ $R2 = 0V$ $R3 = V_{cc}$ $R4 = 0V$
the third row is enabled and the others are disabled, so the $C_1 \dots C_4$ read values are referred to $P9 \dots P12$
- $R1 = 0V$ $R2 = 0V$ $R3 = 0V$ $R4 = V_{cc}$
the fourth row is enabled and the others are disabled, so the $C_1 \dots C_4$ read values are referred to $P13 \dots P16$

Thus, the principle is to enable the row containing the pressed button and read the corresponding column wire.

The entire enable sequence (from enabling the first row to enabling the last row) must be completed faster than the user's response time.

This is usually not a problem, because a person has a response time greater than 100 ms; therefore, with a scanning frequency greater than $40 \text{ Hz} \approx \text{row_number}/100 \text{ ms}$, the user does not perceive any delay when pressing the button. To read the keypad, the rows must be enabled sequentially.

However, if two buttons in the same column are pressed at the same time, the rows can become shorted, which may be dangerous for the system.

For this reason, diodes are used when connecting the rows.

The columns are floating when no button is pressed, so pull-down (or pull-up, depending on the keypad reading logic) resistors are required.

1.4 LCD communication

We use the **16x2 LCD Module** (CFAH1602B-TMC-JP) integrated in the **Altera DE2 board** to show the selected wave function name and wave frequency.

The LCD Module has built in a LSI controller, the controller has two 8-bit registers, an **instruction register** (IR) and a **data register** (DR).

The IR stores instruction codes, such as display clear and cursor shift, and address information for display data RAM (DDRAM).

These two registers can be selected with the **register selector** (RS) signal, if it is set to low is possible to write an operation into the IR register, else if the RS signal is set to high the data are written from the DR to the DDRAM.

The R/W pin can be used to read the Busy Flag (BF), if is enabled, or to read data from DDRAM or CGRAM. Since we don't need those functions we leave it always low.

RS	R/W	Operation
0	0	IR write as an internal operation (display clear, etc.)
0	1	Read busy flag (DB7) and address counter (DB0 to DB7)
1	0	Write data to DDRAM or CGRAM (DR to DDRAM or CGRAM)
1	1	Read data from DDRAM or CGRAM (DDRAM or CGRAM to DR)

Figure 1: Operation selection

1.4.1 Character display

An internal Address Counter (AC) is used to increment the DDRAM address each new writing cycle.

The Character Generator ROM (CGROM) generates 5x8 dot or 5x10 dot character patterns from 8 bit ASCII code stored in the DDRAM to be displayed on the screen.

The table Figure 2 shows how CGROM can output a unique set of dots for each character.

Upper 4 bit Lower 4 bit	CG RAM (1)	CG RAM (2)	CG RAM (3)	CG RAM (4)	CG RAM (5)	CG RAM (6)	CG RAM (7)	CG RAM (8)	CG RAM (9)	CG RAM (10)	CG RAM (11)	CG RAM (12)	CG RAM (13)	CG RAM (14)	CG RAM (15)	CG RAM (16)	CG RAM (17)	CG RAM (18)	CG RAM (19)	CG RAM (20)	CG RAM (21)	CG RAM (22)	CG RAM (23)	CG RAM (24)	CG RAM (25)	CG RAM (26)	CG RAM (27)	CG RAM (28)	CG RAM (29)	CG RAM (30)	CG RAM (31)	CG RAM (32)	CG RAM (33)	CG RAM (34)	CG RAM (35)	CG RAM (36)	CG RAM (37)	CG RAM (38)	CG RAM (39)	CG RAM (40)	CG RAM (41)	CG RAM (42)	CG RAM (43)	CG RAM (44)	CG RAM (45)	CG RAM (46)	CG RAM (47)	CG RAM (48)	CG RAM (49)	CG RAM (50)	CG RAM (51)	CG RAM (52)	CG RAM (53)	CG RAM (54)	CG RAM (55)	CG RAM (56)	CG RAM (57)	CG RAM (58)	CG RAM (59)	CG RAM (60)	CG RAM (61)	CG RAM (62)	CG RAM (63)	CG RAM (64)	CG RAM (65)	CG RAM (66)	CG RAM (67)	CG RAM (68)	CG RAM (69)	CG RAM (70)	CG RAM (71)	CG RAM (72)	CG RAM (73)	CG RAM (74)	CG RAM (75)	CG RAM (76)	CG RAM (77)	CG RAM (78)	CG RAM (79)	CG RAM (80)	CG RAM (81)	CG RAM (82)	CG RAM (83)	CG RAM (84)	CG RAM (85)	CG RAM (86)	CG RAM (87)	CG RAM (88)	CG RAM (89)	CG RAM (90)	CG RAM (91)	CG RAM (92)	CG RAM (93)	CG RAM (94)	CG RAM (95)	CG RAM (96)	CG RAM (97)	CG RAM (98)	CG RAM (99)	CG RAM (100)	CG RAM (101)	CG RAM (102)	CG RAM (103)	CG RAM (104)	CG RAM (105)	CG RAM (106)	CG RAM (107)	CG RAM (108)	CG RAM (109)	CG RAM (110)	CG RAM (111)	CG RAM (112)	CG RAM (113)	CG RAM (114)	CG RAM (115)	CG RAM (116)	CG RAM (117)	CG RAM (118)	CG RAM (119)	CG RAM (120)	CG RAM (121)	CG RAM (122)	CG RAM (123)	CG RAM (124)	CG RAM (125)	CG RAM (126)	CG RAM (127)	CG RAM (128)	CG RAM (129)	CG RAM (130)	CG RAM (131)	CG RAM (132)	CG RAM (133)	CG RAM (134)	CG RAM (135)	CG RAM (136)	CG RAM (137)	CG RAM (138)	CG RAM (139)	CG RAM (140)	CG RAM (141)	CG RAM (142)	CG RAM (143)	CG RAM (144)	CG RAM (145)	CG RAM (146)	CG RAM (147)	CG RAM (148)	CG RAM (149)	CG RAM (150)	CG RAM (151)	CG RAM (152)	CG RAM (153)	CG RAM (154)	CG RAM (155)	CG RAM (156)	CG RAM (157)	CG RAM (158)	CG RAM (159)	CG RAM (160)	CG RAM (161)	CG RAM (162)	CG RAM (163)	CG RAM (164)	CG RAM (165)	CG RAM (166)	CG RAM (167)	CG RAM (168)	CG RAM (169)	CG RAM (170)	CG RAM (171)	CG RAM (172)	CG RAM (173)	CG RAM (174)	CG RAM (175)	CG RAM (176)	CG RAM (177)	CG RAM (178)	CG RAM (179)	CG RAM (180)	CG RAM (181)	CG RAM (182)	CG RAM (183)	CG RAM (184)	CG RAM (185)	CG RAM (186)	CG RAM (187)	CG RAM (188)	CG RAM (189)	CG RAM (190)	CG RAM (191)	CG RAM (192)	CG RAM (193)	CG RAM (194)	CG RAM (195)	CG RAM (196)	CG RAM (197)	CG RAM (198)	CG RAM (199)	CG RAM (200)	CG RAM (201)	CG RAM (202)	CG RAM (203)	CG RAM (204)	CG RAM (205)	CG RAM (206)	CG RAM (207)	CG RAM (208)	CG RAM (209)	CG RAM (210)	CG RAM (211)	CG RAM (212)	CG RAM (213)	CG RAM (214)	CG RAM (215)	CG RAM (216)	CG RAM (217)	CG RAM (218)	CG RAM (219)	CG RAM (220)	CG RAM (221)	CG RAM (222)	CG RAM (223)	CG RAM (224)	CG RAM (225)	CG RAM (226)	CG RAM (227)	CG RAM (228)	CG RAM (229)	CG RAM (230)	CG RAM (231)	CG RAM (232)	CG RAM (233)	CG RAM (234)	CG RAM (235)	CG RAM (236)	CG RAM (237)	CG RAM (238)	CG RAM (239)	CG RAM (240)	CG RAM (241)	CG RAM (242)	CG RAM (243)	CG RAM (244)	CG RAM (245)	CG RAM (246)	CG RAM (247)	CG RAM (248)	CG RAM (249)	CG RAM (250)	CG RAM (251)	CG RAM (252)	CG RAM (253)	CG RAM (254)	CG RAM (255)
----------------------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------	--------------------

Figure 2: Character Generator ROM pattern

1.4.2 Timing Characteristics

It is essential to respect the timing characteristic of LCD module to correctly complete a write cycle without issues.

As we can see in Figure 3 the RS and R/W signals need to be stable before the E signal is set to high and the enable pulse needs to be sufficiently wide to collect the data, then after disabling E an hold time is required to complete the cycle.

In our case a fast display refresh frequency is not required, so it is possible to use delays way larger than the stated times reported by the datasheet, nonetheless the correct sequence of the chip enable (E) needs to be used.

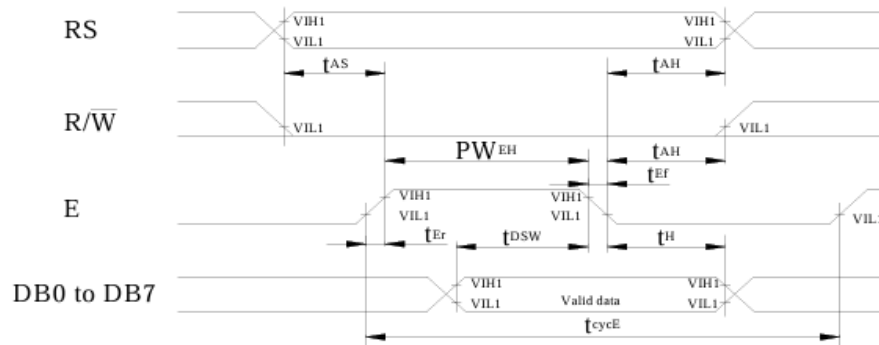


Figure 3: Write Operation

2 Description of the architecture

To analyze the frequency generator designed for this project, we can start from the structure of three main blocks

- **Human-Machine interface**

To select the desired output wave at the right frequency, this generator provides an LCD screen and a keypad.

The keypad is used to write the frequency manually and to select the output waveform, information that is shown on the LCD screen.

- **Digital waveform generator**

The digital waveform generator uses the DDS technique to create the digital output.

The representation of the output value is C1 to be compatible with the DAC, and to increase the number of waveforms available, vertical and horizontal mirroring is selectable.

- **Digital analog converter**

The DDS module is designed to have a 12 *bit* output digital value, but for the project a is used a VGA DAC that has three 10 *bit* channels.

Only the most significant bits of the module have been used.

2.1 Specifications

The frequency generator designed is capable to generate three different output wave and the mirroring combinations of them, with a total of 8 valid combinations (some combinations are valid but not useful)

- **Square wave**

available only with vertical and horizontal mirroring enabled, for every other combinations of vertical and horizontal mirroring the output value is constant at 0xFF (hex notation), or result in a square wave with two time the selected frequency.

- **Triangular wave**

available with vertical and horizontal mirroring enabled. If the vertical and the horizontal mirroring is disabled the output is a **sawtooth wave**, instead if only the horizontal mirroring is enabled, the output is an **alternating symmetrical sawtooth wave**

- **Sinusoidal wave**

available with vertical and horizontal mirroring enabled. If only the vertical mirroring is enabled the waveform is a **rectified sinusoidal wave**. If only the horizontal mirroring is enabled the waveform is an **alternating symmetrical quarter-sine wave**. If both of the mirroring are disabled the waveform is a **quarter-sine wave**

The output frequency can be selected with an LCD screen and a keypad and can go from 1 *Hz* to 1 *MHz* without using external memory for the LUT.

$$\max(f_{out}) = \frac{\max(f_{clk})}{LUT_DIM / \max(f_w) \cdot 4} = \frac{35 MHz}{2^{10} / 2^7 \cdot 4} \simeq 1.09 MHz$$

The DAC used is a 10 *bit* VGA DAC with three channels synchronized with a common clock at the maximum frequency of 100 *MHz*.

The maximum clock frequency is 35 *MHz* because the project is synchronized with a common clock source at a frequency of 70 *MHz* that passes through a frequency divider, which dynamically divides the frequency with a division coefficient ranging from 2 to 1000000.

2.2 Module description

As seen in the image below, the project is made of different modules

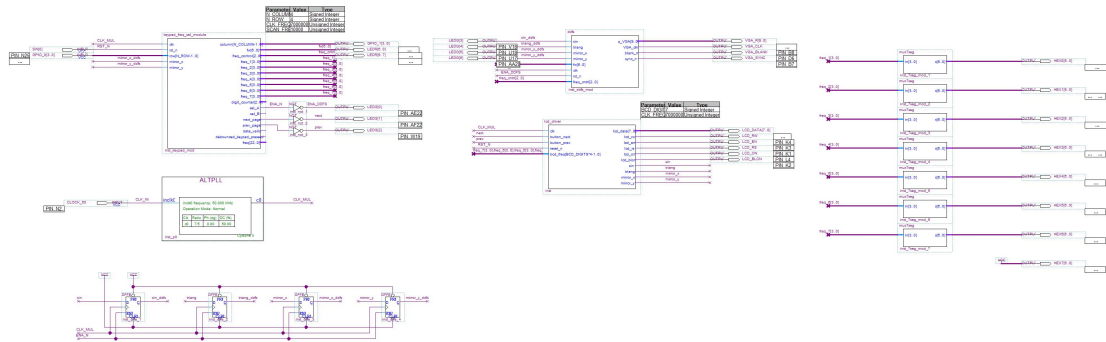


Figure 4: Top level entity symbol schematic

2.2.1 DDFS module

The DDFS module generates the bit-stream corresponding to the samples of the output waveform at the desired frequency.

The samples are synchronized with a common clock signal, and the frequency of the output wave can be selected through the `fw` and `freq_ctrl` inputs.

This module has been designed in two different ways:

- the first version is intended to be connected to the **ADV7123 DAC**, which is a *10bit* synchronous VGA DAC
- the second version is intended to be connected to a *12bit* asynchronous R-2R ladder DAC.

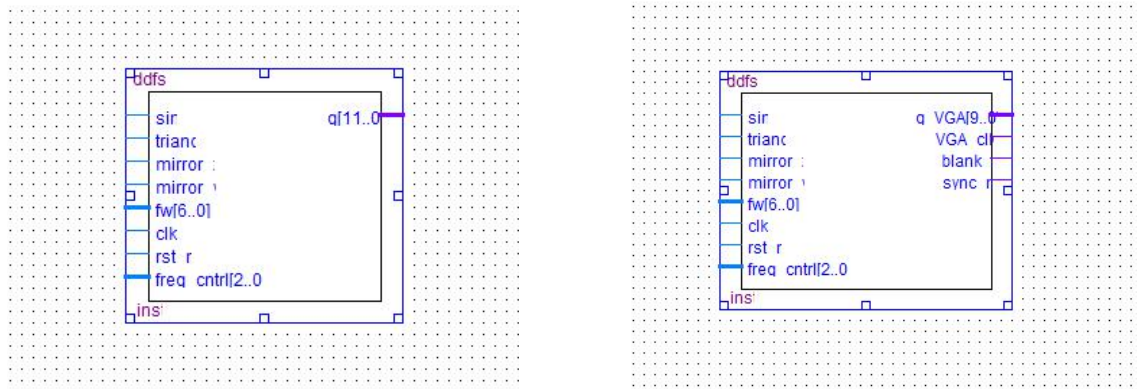


Figure 5: DDFS module

I/O interface

The ports used to connect this module to other modules or to the external pins of the FPGA are the following:

- `sin`, `tri`
used to select the output waveform. If `sin` and `tri` are both zero, the output signal is a square wave. The sinusoidal output has priority.
- `mirror_x`, `mirror_y`
used to manually select the symmetry of the output wave. `mirror_x` is used to enable the horizontal symmetry of the wave, and `mirror_y` to enable the vertical symmetry. For every symmetry pin enabled, the period of the output wave doubles because the wave needs to be repeated mirrored.

- **f_w**
is the value of the frequency word. It is composed of *7bits* and is unsigned, so the maximum value is $2^7 - 1 = 127$. The instantiated LUT is a (12×1024) bit memory where the address value goes from 0 to 1023, and the stored samples are *12bit* values. The number of samples sent to the DAC after a complete scan cycle are $8 < \frac{1024}{f_w+1} < 1024$.
- **clk, rst_n**
are the clock and the synchronous reset pins.
- **freq_cntrl**
is a select input used to scale the clock frequency by a fixed value. It is a *3bit* input pin, and the scalable frequency values are $\frac{f_{clk}}{1.000.000}, \frac{f_{clk}}{100.000}, \frac{f_{clk}}{10.000}, \dots, \frac{f_{clk}}{100}, \frac{f_{clk}}{10}, \frac{f_{clk}}{2}$. The total scalable frequencies are 7, and they are associated with the values of **freq_cntrl** from 0 to 6.
- **q or q_VGA, blank, sync_n, VGA_clk**
are the digital output values. For the first version, q is a *12bit* value synchronized with the divided value of the clock.
For the VGA_DAC, the q_VGA output is a *10bit* bus, so the 2 least significant bits are truncated. The VGA_DAC also needs to be synchronized with a clock to sample the values, so the output of the frequency divider has been connected to the **phase accumulator** and **VGA_clk**. The **blank** and **sync_n** signals are connected to 1 because they are not used.

Verilog code

In the first part of the module is declared a *12bits* register where is stored a quarter of the wave to show. The content of the register depend on the output waveform selected.

```

1  reg [11:0] q_tmp;
2  localparam HALF_ADDRESS = 2** (ADDR_WIDTH - 1);
3  always @(*) begin
4
5      if(sin)
6          q_tmp = lut_out;          //sinusoidal output
7      else if(triang)
8          q_tmp = addr*(12'hfff/(MAX_LUT-1));    //triangular output
9      else
10         q_tmp = 12'hfff;          //square output
11
12  end

```

Listing 1: Selection of the output waveform

if the sinusoidal wave is selected, the register is loaded with the output of the LUT. The triangular wave is calculated multiplying the address value of the LUT with a coefficient that allow it to go from 0x000 to 0xFFFF that is the full range of the digital value.

The quarter triangular-wave is calculated as follow

$$\text{out} = \text{address} \cdot \frac{0xFFFF}{\text{max}(\text{address})} = \text{address} \cdot \frac{0xFFFF}{\text{MAX_LUT} - 1}$$

A quarter of a square-wave is a constant value, so in this case the output is 0xFFFF.

In the second part is defined the **phase accumulator**, the counter need to be four times larger than the maximum address of the LUT, because in the worst case where are enabled both of the symmetry the period of the wave is $4 \cdot T$ where T is the period of the quarter wave stored in the LUT. The LUT have a 1 clock cycle latency after the read request, for this reason has been defined a counter re-timed with a clock cycle synchronized with the LUT output.

```

1 //retiming register for address counter
2 reg [ADDR_WIDTH+1:0] cont_rt;
3 always @(posedge clk_div)
4     cont_rt <= cont;
5
6 //calculate address
7 always @(*) begin
8
9     //not mirrored case
10    addr = cont%(MAX_LUT);
11
12    //y mirrored output
13    if(mirror_y)
14        //second quarter or fourth quarter
15        if(
16            (cont > (MAX_LUT-1)) && (cont <= (2*MAX_LUT-1)) ||
17            (cont > (3*MAX_LUT-1)) && (cont <= (4*MAX_LUT-1))
18        )
19            addr = (MAX_LUT-1) - cont%(MAX_LUT);
20
21    end

```

Listing 2: Address calculation with vertical mirroring

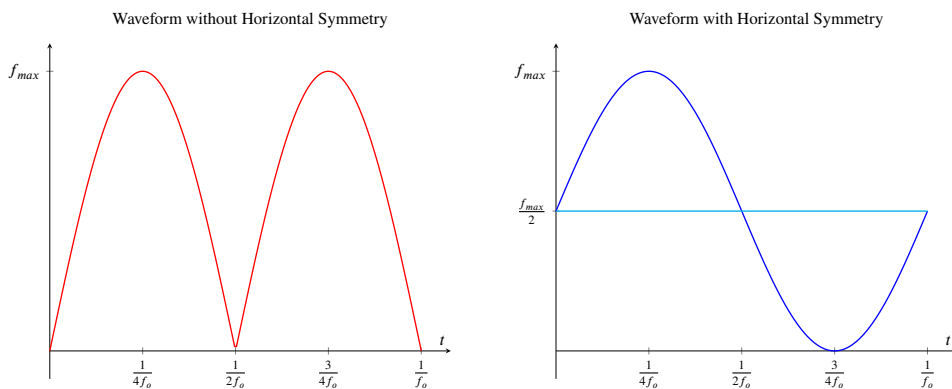
When mirroring is disabled, the address is equal to $|\text{counter}|_{\text{MAX_LUT}}$.

To simplify hardware synthesis, MAX_LUT is better chosen as a power of two; in this case, the modulo operation becomes simply a truncation of the most significant bits.

When vertical mirroring is enabled, the circuit checks the address counter to determine the quarter of the wave to display, and when the signal is in the third or fourth quarter, the address sweeps from the last entry of the LUT to the first one.

In the last part of the module, the output with horizontal symmetry is calculated.

To allow the output to be represented with 12bits under horizontal symmetry, starting from a 12bit sample, the output is divided by two and summed with an offset constant equal to half of the total output swing.



The operation performed to the output is $\text{out} = \frac{q}{2} + \frac{f_{max}}{2} = \frac{q}{2} + \text{HALF_DATA}$. When the horizontal symmetry is enabled the output must be inverted, so the output is $\text{out} = -\frac{q}{2} + \frac{f_{max}}{2} = -\frac{q}{2} + \text{HALF_DATA}$.

```

1  //calculate the output
2  reg [ADDR_WIDTH+1:0] act_cont;
3  localparam HALF_DATA = 2**(DATA_WIDTH-1);
4  always @(*) begin
5
6      //not mirrored case
7      q = HALF_DATA + (q_tmp >> 1) - 1;
8
9      //select the correct counter for the wave we are using
10     if(sin)
11         act_cont = cont_rt;
12     else
13         act_cont = cont;
14
15     if(mirror_x && mirror_y) begin
16         //third quarter or fourth quarter
17         if(
18             ((act_cont > (2*MAX_LUT-1)) && (act_cont <= (3*MAX_LUT-1))) ||
19             ((act_cont > (3*MAX_LUT-1)) && (act_cont <= (4*MAX_LUT-1)))
20         )
21             q = HALF_DATA - (q_tmp >> 1) - 1;
22
23     end else if(mirror_x) begin
24         //second quarter or fourth quarter
25         if(
26             ((act_cont > (MAX_LUT-1)) && (act_cont <= (2*MAX_LUT-1))) ||
27             ((act_cont > (3*MAX_LUT-1)) && (act_cont <= (4*MAX_LUT-1)))
28         )
29             q = HALF_DATA - (q_tmp >> 1) - 1;
30
31     end
32
33 end

```

Listing 3: Output calculation with horizontal mirroring

The horizontal symmetry needs to be applied during the second and fourth quarters when vertical symmetry is not enabled.

When vertical symmetry is enabled, the period of the wave is $2 \cdot T$, and horizontal symmetry must be applied in the third and fourth quarters.

To calculate the address, a global counter is used, and is performed the modulo operation with base MAX_LUT.

The problem with this technique is that when the address moves from one quarter to another, if f_w is greater than one, the address can start from a value bigger than zero.

For example, if MAX_LUT is 1024 and $cont$ is 1000, with f_w equal to 600, the next value of $cont$ is $1000 + 600 = 1600$.

This means that address is equal to $cont$ when it is 1000, but at the next clock cycle it becomes $|1600|_{1024} = 576$.

Therefore, the address starts counting from 0, and in the next quarter of the wave it starts from 576.

This behavior causes the output waveform to be slightly different from period to period.

To avoid this, when the address is about to overflow, the counter is set to a multiple of MAX_LUT , so that at the next cycle the address starts from $|N \cdot 1024|_{1024} = 0$.

```

1  if(cont < MAX_LUT-1) begin
2      if(cont + (fw+1) >= MAX_LUT-1)
3          cont <= MAX_LUT;
4      else
5          cont <= cont + (fw+1);
6  end
7  else if(cont < 2*MAX_LUT-1) begin
8      if(cont + (fw+1) >= 2*MAX_LUT-1)
9          cont <= 2*MAX_LUT;
10     else
11         cont <= cont + (fw+1);
12 end
13 else if(cont < 3*MAX_LUT-1) begin
14     if(cont + (fw+1) >= 3*MAX_LUT-1)
15         cont <= 3*MAX_LUT;
16     else
17         cont <= cont + (fw+1);
18 end
19 else if(cont < 4*MAX_LUT-1) begin
20     if(cont + (fw+1) >= 4*MAX_LUT-1)
21         cont <= 0;
22     else
23         cont <= cont + (fw+1);
24 end
25 else
26     cont <= 0;

```

Listing 4: Calculation of counter for DDFS address

Sub-modules

Inside this module are instantiated two other modules

- **Programmable frequency divider** to scale the clock frequency to lower values
- **Look Up Table** to store the digital samples for sinusoidal output

Programmable frequency divider

We implemented a scalable frequency divider controlled by the `freq_ctrl` 3bit signal to divide the output frequency with a coefficient ranging from 2 to 1000000.

This is useful to maintain a decent quality of the output wave, even with higher values of the frequency word. With this method, using an input frequency of $50MHz$, we can select the output frequencies of $25MHz$, $5MHz$, $500kHz$, $50kHz$, $5kHz$, $500Hz$, and $50Hz$.

A 20bit counter is used, and the output is connected to the MSB of the counter.

The output clock is then generated with a 0.5 duty cycle by starting the counter when the MSB is 0 and, after half of the counter period, the MSB becomes 1; then the counter resets to the start.

When `freq_ctrl` changes, the `start_cnt` and `end_cnt` registers are updated, and thus the counter restart condition also changes, avoiding conflicts.

```
1  always @(posedge clk_in) begin
2      if(rst_n == 1'b0) begin
3          count <= start_cnt;
4      end else if(count >= (end_cnt-1'b1))begin
5          count <= start_cnt;
6      end else begin
7          count <= count + 1'b1;
8      end
9
10 end
11
12 always @(*) begin
13
14     case (freq_ctrl)
15
16         0:begin
17             start_cnt = 20'h80000-1;
18             end_cnt = start_cnt+2;
19         end
20         1:begin
21             start_cnt = 20'h80000-10/2;
22             end_cnt = start_cnt + 10;
23         end
24         .
25         .
26         .
27
28         6:begin
29             start_cnt = 20'h80000-1000000/2;
30             end_cnt = start_cnt + 1000000;
31         end
32         default:begin
33             start_cnt = 20'h80000-1000000/2;
34             end_cnt = start_cnt + 1000000;
35         end
36     endcase
37
38     clk_out = count[19];
39
40
41 end
```

Listing 5: Programmable frequency divider

Look Up Table

The LUT is a pre-initialized ROM instantiated inside the FPGA with the samples of a quarter of a sinusoidal wave.

The samples stored in the ROM are $12bit$ samples calculated as follows

$$\text{sample}[n]_{12bit} = \sin\left(\frac{n}{\text{LUT_DIM} - 1} \cdot \frac{\pi}{2}\right)_{12bit}$$

The LUT_DIM is choose to be 2^{10} , so the LUT is a $(12 \times 1024)bit$ ROM inside the FPGA.

2.2.2 Keypad frequency select module

This module reads the button pressed on the keypad every N clock cycles.

The keypad is used to enter the frequency of the wave, and the output of this module is the frequency in BCD or **binary-coded decimal** representation.

Inside the module, there is a converter network that converts the frequency value into a combination of `f_w` and `freq_cntrl`, which are the inputs of the DDFS module used to control the output wave frequency.

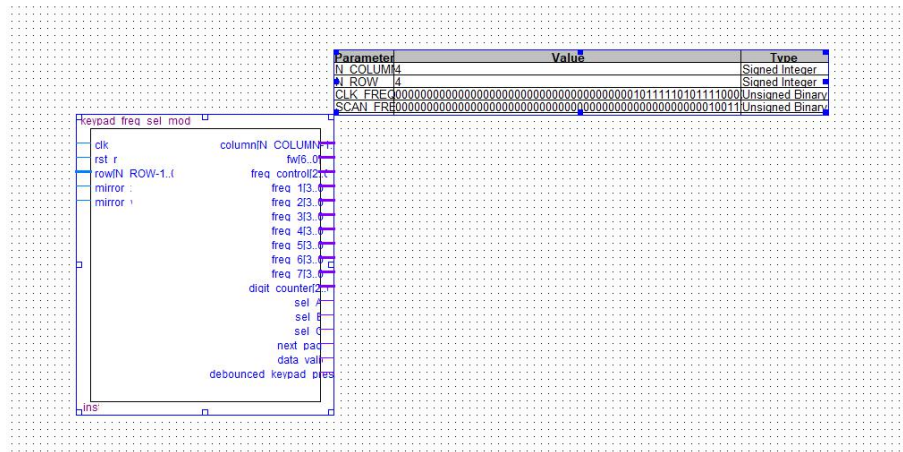


Figure 6: Keypad frequency select module

I/O interface

The ports used to connect this module to other modules or to the external pins of the FPGA are the following:

- `clk, rst_n`
are the clock and the synchronous reset pins.
- `row`
this is the input pin from the keyboard.
These pins need to be connected to the internal pullup of the FPGA, because the keypad matrix, when a button is not pressed, leave the line floating.
Every bit of this word is connected to a row line of the keypad.
- `mirror_x, mirror_y`
these are the mirroring pins of the output wave; if the output wave is mirrored, the LUT must be read two or four times (depending on how many mirroring effects are applied to the wave) every period.
This means that the `f_w` and `freq_control` values depend on the value of `mirror_x` and `mirror_y`.
- `column`
this output pin is connected to the keypad, and is used to enable the column before reading a row.
By enabling a column and reading the row values is possible to determine which button is pressed.
- `f_w, freq_control`
this outputs goes to the DDFS module to set the frequency of the output wave.
- `freq_1, freq_2, ... ,freq_7`
this is the 7 digits of the output frequency expressed in BCD representation.
- `digit_counter`
this output represent the number of the frequency digits.
- `sel_A, sel_B`
these outputs are toggled every time the A and B buttons of the keypad are pressed.
This can be useful because the VGA DAC has three *10bit* inputs, so it is possible to instantiate two DDFS modules connected to each DAC input, and these pins can individually enable each DDFS module.

- `next_page`, `prev_page`
this output is connected to LCD module to change the wave to be visualized.
- `data_valid`, `debounced_keypad_pressed`
are test outputs used for debugging.

Parameters

The parameters of this module are:

- `N_COLUMN`, `N_ROW`
are the number of rows and column of the keypad module used
- `SCAN_FREQ`
this is the frequency used to perform the keypad scanning.
If the scan operation is performed too rapidly, the parasitic capacitance of the buttons, in series with the pull-up resistors, can create reading problems.
The scan frequency value used is $10kHz$.
- `CLK_FREQ`
is the clock frequency of the clock connected to this module.
Is used to synchronize the timers for the debounce circuit and the scanning frequency of the keypad.

Verilog code

The module is made of a single sequential network that checks every clock cycle whether the `data_valid` signal (from the sub-module that reads the keypad buttons) is high, and reads which button is pressed in that case.

The first problem is that the button bounces significantly before stabilizing in one position.

For this reason, a debounce circuit has been implemented, with a timer that waits for approximately $100ms$ after the button is pressed before considering the button as pressed.

```

1      always @(posedge clk) begin
2          if(rst_n == 0) begin
3              //reset condition
4              debounce_cont <= 0;
5              debounced_keypad_pressed <= 1'b0;
6          end else if(clk_ena == 1) begin
7              if(out_keys == 16'd0) begin
8                  debounce_cont <= 0;
9                  debounced_keypad_pressed <= 1'b0;
10                 out_keys_sync <= out_keys_sync;
11             end else begin
12                 debounce_cont <= debounce_cont + 1;
13                 if(debounce_cont >= DEBOUNCE_CYCLE) begin
14                     debounce_cont <= DEBOUNCE_CYCLE;
15                     debounced_keypad_pressed <= 1'b1;
16                     out_keys_sync <= out_keys;
17                 end
18             end
19         end else begin
20             debounce_cont <= debounce_cont;
21             debounced_keypad_pressed <= debounced_keypad_pressed;
22             out_keys_sync <= out_keys_sync;
23         end
24     end

```

Listing 6: Debounce circuit

The second problem is that a button is usually pressed for too long; for this reason, a flag has been instantiated to check if the button has been released.

This flag makes it possible to perform an action only once, even if the button remains pressed for many clock cycles.

```

1      if(data_valid == 1) begin
2          if(debounced_keypad_pressed == 0) begin
3              //flag reset
4              button_pressed <= 1'b0;
5          end
6          if(debounced_keypad_pressed == 1 && button_pressed == 0) begin
7              .
8              .
9              .
10             .
11             .
12             //flag set
13             button_pressed <= 1'b1;
14         end
15     end

```

Listing 7: Flag to avoid multiple operation with a single press

Now we have to define the behavior of the machine upon the press of each button.

When multiple buttons are pressed simultaneously, the machine does nothing by default.

When a number is pressed, the machine shifts the digits of the frequency number to the left and writes the pressed number in the place of the least significant digit.

If a new number have to be selected, the flag number_not_sel is high and the number become the digit pressed.

```

1      // ---- NUMBER KEY PRESSED ---- //
2
3      (16'd1 << 0): begin
4          if(number_not_sel == 1) begin
5              {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1} <= 1;
6              digit_counter <= 3'd1;
7              number_not_sel <= 1'b0;
8          end else if(digit_counter < 7) begin
9              //shift right and write the first digit
10             {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1} <=
11             {freq_6, freq_5, freq_4, freq_3, freq_2, freq_1, 4'd1};
12             //increase the number of digit
13             digit_counter <= digit_counter + 1;
14         end
15     end
16
17     .
18     .
19     .
20
21     // ---- DEFAULT CONDITION ---- //
22
23     default: begin
24         {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1} <=
25         {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1};
26         digit_counter <= digit_counter;
27     end

```

Listing 8: Number digit pressed on the keypad

The letter buttons toggle the sel_A and sel_B outputs, the C and D buttons of the keypad are used to send the next page or previous page signals to the LCD module.
The last two buttons of the keypad are used to confirm the frequency value written or to erase the digits.

```

1      // ---- CONFIRM OR ERASE KEY PRESSED ---- //
2
3      (16'd1 << 14): begin
4          if(digit_counter > 0) begin
5              //digit erased, shift left the digit
6              {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1} <=
7              {4'd0, freq_7, freq_6, freq_5, freq_4, freq_3, freq_2};
8
9              //decrease the number of digit
10             digit_counter <= digit_counter - 1;
11         end
12     else begin
13         {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1} <=
14         {BCD_7, BCD_6, BCD_5, BCD_4, BCD_3, BCD_2, BCD_1};
15         //store the previous number of digit
16         digit_counter <= digit_counter_BCD;
17         number_not_sel <= 1'b1;
18     end
19 end
20
21 (16'd1 << 12): begin
22     //frequency confirmed, save the frequency value
23     {BCD_7, BCD_6, BCD_5, BCD_4, BCD_3, BCD_2, BCD_1} <=
24     {freq_7, freq_6, freq_5, freq_4, freq_3, freq_2, freq_1};
25     //load the actual number of digit
26     digit_counter_BCD <= digit_counter;
27     number_not_sel <= 1'b1;
28 end

```

Listing 9: Confirm or erase button pressed on the keypad

When a value is confirmed, the flag number_not_sel is set to one, so when a digit is pressed, the new number starts to be written from the least significant digit.
When a digit is erased, the number is shifted to the right and the most significant digit become a zero.

Sub-modules

Inside this module are instantiated three other modules

- **Keypad reader module** to read the button values of the keypad.
- **BCD to C1 converter** to convert the frequency value from BCD to C1 representation.
- **DDFS frequency converter** to convert the C1 frequency value into a combination of f_w and $freq_control$ used to control the DDFS module.

Keypad reader module

This module uses a shift register connected to the column lines to enable a different column every clock cycle.

The logic is reversed: a column is considered enabled when it is pulled low, and a row is considered active when the line value is low.

This choice was made because the FPGA GPIOs can be defined with internal $25k\Omega$ pull-up resistors but no internal pull-downs.

After N_{col} clock cycles, where N_{col} is the number of columns of the keypad, the controller pulls up the `data_valid` pin to indicate that the output of the module contains the pressed button.

Every clock cycle, a counter called `i_column` increases, and when the value of the counter reaches N_{col} , it activates `data_valid`.

At each clock cycle, a different column is enabled, and the value of the pressed row is stored in an output vector called `out_keys`.

The structure of the output value is a `bidimensional-vector`, that has been implemented using a technique called **array flattening**.

The array flattening technique consists of dividing the array into consecutive sub-arrays representing the rows of the matrix, where the columns correspond to the values inside the sub-arrays.

The flattened array contains $N_{col} \cdot N_{row}$ elements, and the index of each element is calculated from the i_{col} and i_{row} indices using the following formula $i = N_{row} \cdot i_{col} + i_{row}$.

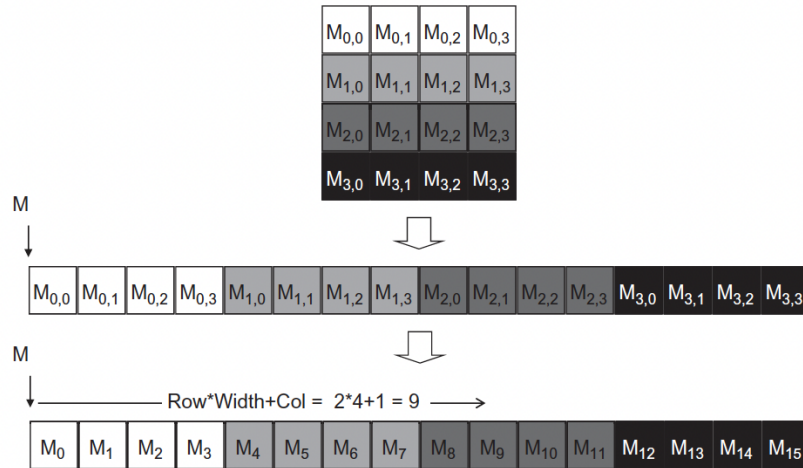


Figure 7: Array flattening technique

The shift register connected to the columns is implemented with a combinatorial network that reads the value of `i_column` and pulls down the corresponding line.

To prevent the module from sampling the row values too close to the change of column enable, the `i_column` values range from 0 to $2 \cdot N_COLUMN$.

When `i_column` is odd, the column is enabled; when it is even, the row is read.

By doing this, the reading operation occurs one clock cycle after the column enabling operation.

```

1  //assign the output values at the column of the keypad
2  integer i;
3  always @(*) begin
4      //default values
5      i = 0;
6      column = ~0;
7
8      //check if i_column is in range
9      if(i_column <= 2*N_COLUMN) begin
10         for(i=0; i<N_COLUMN; i=i+1) begin
11             if(i == (i_column >> 1)) begin
12                 column[i] = 0;
13             end else begin
14                 column[i] = 1;
15             end
16         end
17     end
18
19 end

```

Listing 10: Shift register implementation

Another combinatorial network updates a flattened vector that contains the button pressed for the currently enabled column.

```

1  //read the button values from the keypad
2  integer i_col, i_row;
3  always @(*) begin
4      //default values
5      i_col = 0;
6      i_row = 0;
7
8      //read button cycles
9      for(i_col=0; i_col<N_COLUMN; i_col=i_col+1) begin
10         for(i_row=0; i_row<N_ROW; i_row=i_row+1) begin
11             if(row[i_row] == 0 && i_col == (i_column >> 1)) begin
12                 key_pressed[i_col*N_ROW + i_row] = 1;
13             end else begin
14                 key_pressed[i_col*N_ROW + i_row] = 0;
15             end
16         end
17     end
18
19 end

```

Listing 11: Button reading implementation

Every two clock cycle, the output vector is updated with the button pressed for each column and after $2 \cdot N_{col}$ cycles the reading cycle is done and data output is valid.

The two main issues encountered when reading the keypad are:

- The button press event is asynchronous.
- The scan frequency cannot be too high due to the parasitic capacitance of the buttons and the pull-up resistors.

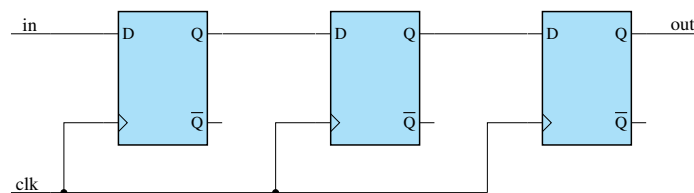
For the first issue, the module provides a flip-flop chain directly connected to the keypad inputs, reducing the probability of metastability problems.

```

1 //latch chain to reduce metastability problems
2 reg [N_ROW-1:0] row_1, row_2, row_sync;
3 always @(posedge clk) begin
4     if(rst_n == 0) begin
5         //reset condition
6         row_1 <= 0;
7         row_2 <= 0;
8         row_sync <= 0;
9     end
10    else begin
11        row_sync <= row_2;
12        row_2 <= row_1;
13        row_1 <= row;
14    end
15 end

```

Listing 12: flip-flop chain to reduce metastability problem occur



To solve the second issue, the module includes a clock enable signal generated by an internal counter. Each time the clock enable signal is asserted, the synchronized operations are performed. The clock enable signal is also forwarded to the keypad frequency selection module.

```

1 //clock enable generator
2 localparam FREQ_DIV = (CLK_FREQ/SCAN_FREQ>3) ? CLK_FREQ/SCAN_FREQ : 3;
3 integer clock_div_cont;
4 always @(posedge clk) begin
5     if(rst_n == 0) begin
6         //reset condition
7         clock_div_cont <= 0;
8         clk_ena <= 1'b0;
9     end
10    else begin
11        clock_div_cont <= clock_div_cont + 1;
12        clk_ena <= 1'b0;
13        if (clock_div_cont >= FREQ_DIV) begin
14            clock_div_cont <= 0;
15            clk_ena <= 1'b1;
16        end
17    end
18 end

```

Listing 13: Clock enable signal generation

The flip-flop chain introduces a delay, as the output data is delayed with respect to the input data by three clock cycles.

For this reason, the `FREQ_DIV` parameter is defined to be greater than three, ensuring that the output delay introduced by the chain is absorbed before the next reading operation.

BCD to C1 converter

This module takes as input the value to be converted from BCD representation and converts it into a C1 representation value.

To perform this operation, the value must be calculated as follows

$$n_{C1} = \sum_{i=0}^{n_digit} a_i \cdot 10^i$$

This operation instantiates many multipliers, which are hardware-expensive; for this reason, every power of 10 is decomposed into a sum of powers of two

$$n_{C1} = \sum_{i=0}^{n_digit} a_i \cdot 10^i = \sum_{i=0}^{n_digit} \left[a_i \cdot \left(\sum_j b_j \cdot 2^j \right) \right] \quad b_j \in \{1, 0\}$$

Doing this transforms every multiplication into a combination of additions and left shifts, which are less expensive in terms of hardware primitives instantiated on the FPGA.

DDFS frequency converter

This module reads the frequency value to be sent to the DDFS module and converts it into a combination of `f_w` and `freq_control`, which are the frequency-control inputs of the DDFS module.

The output frequency of the DDFS module is calculated as follows

$$f_{out} = \frac{f_{clk}}{N} \cdot \frac{f_w + 1}{LUT_DIM} \quad f_w \in [0, 127]$$

where $\frac{f_{clk}}{N}$ is the divided clock frequency from the frequency divider, and f_w is the frequency word. The output frequency range for each divided frequency is (in our case $LUT_DIM = 2^{10} = 1024$)

$$f_{out} \in \left[\frac{f_{clk}}{N \cdot LUT_DIM}, \frac{f_{clk} \cdot 128}{N \cdot LUT_DIM} \right] \xrightarrow{LUT_DIM=1024} \left[\frac{f_{clk}}{N \cdot 1024}, \frac{f_{clk}}{N \cdot 8} \right]$$

For any selected output frequency, the module chooses a value of N such that the desired frequency lies within the range $\left[\frac{f_{clk}}{N \cdot 1024}, \frac{f_{clk}}{N \cdot 8} \right]$.

Once the value of N has been chosen, to calculate f_w the formula is the following

$$f_w = f_{out} \cdot \frac{N \cdot LUT_DIM}{f_{clk}} = f_{out} \cdot \frac{N \cdot 2^{10}}{f_{clk}}$$

The problem with this formula is that, even if the value of f_{clk}/N is pre-calculated, the synthesizer instantiate a divider to do the f_{out}/f_{clk} operation, and this have to be done for every frequency range, so the divider instantiated are more than one.

Instantiating a divider is challenging because it results in a large combinational network and, unless the operation is pipelined, it significantly reduces the maximum operating frequency of the design.

Another drawback of instantiating a divider is that it occupies a considerable amount of device resources.

An alternative approach is to use the following formula

$$f_w = \frac{2^{64}}{f_{clk}} \cdot \frac{f_{out} \cdot N}{2^{54}} = K \cdot \frac{f_{out} \cdot N}{2^{54}}$$

using this method, K can be pre-calculated, and the division for 2^{54} can be done with a shift left operation. In this way, the compiler can replace the division with internal multipliers, thus avoiding the instantiation of hardware dividers.

Multipliers are also large combinatorial networks, but they are smaller and faster compared to dividers.

2.3 LCD module

In this section we have a description of the driving mechanism used for the LCD module.

The goal is to display a character sequence relative to the selected wave function name and its frequency, and to change each output wave (and its displayed name) using two buttons to move to the previous or next wave function.

To store the character to display a LUT is used, containing, for each entry, a 9 bit code addressable with a page address and a character address as we will see later on.

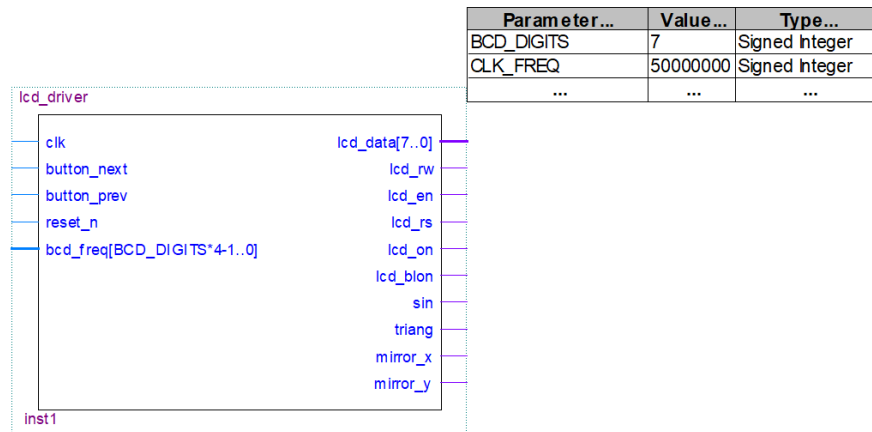


Figure 8: LCD driver module

I/O interface

The ports used to connect this module to other modules or to the external pins of the FPGA are the following:

- **clk, reset_n**
are the clock and the synchronous reset pins
- **button_next, button_prev**
these are connected to the input buttons used for cycling through each page and so each wave function to display
- **bcd_freq**
this input is received from the "Keypad frequency select module" and it is the BCD converted frequency to display on the screen. Each decimal digit to display needs 4 bit to be represented in BCD, so the parameter BCD_DIGITS is used to instantiate the correct bit value for the digits we want to display
- **lcd_data**
this is the 8 bit data bus connected to the LCD controller used to write instructions in the IR (instruction register) or the 8 bit ASCII code the DR (data register)
- **lcd_en**
this pin enable the writing cycle from the data bus to the internal registers of the LCD controller (IR or DR)
- **lcd_rs**
this pin is used to select the writing on the IR or DR (register select)
- **lcd_on**
this pin control the power supply of the LCD and it is always high
- **lcd_blon**
this pin control the backlight of the LCD and it is always on
- **sin, triang**
these pins are sent to the DDFS module to enable the output of the selected wave function

- mirror_x ,mirror_y
these pins are sent to the DDFS module to change the mirroring options of the waveform

Parameters

The parameters of this module are:

- BCD_DIGITS
are the number of frequency digit to display on the module.
- CLK_FREQ
is the clock frequency of the clock connected to this module.
Is used to synchronize the timers for the debounce circuits and to respect the timing constrain for the communication with the LCD.

Verilog code

The LCD module is composed of 2 Synchronous Finite State Machines. One operates to display the name of the selected function in the first row of the screen, while the other continuously displays the selected frequency in the second row of the display when the other FSM is waiting for a button pressure (and release) to change page.

To implement two different FSM in the same Verilog code we need to multiplex the shared registers between the two FSM, in our case the registers to be multiplexed are the ones responsible for the LCD driving: lcd_rs, lcd_data, lcd_en.

At the end of this combinational always block is present the extrapolation of each BCD number (bcd_number) from the input bit vector (bcd_freq), using a reverse indexing strategy.

```

1      always @(*) begin
2          if( state != WAIT_BUTTON_SHORT && state != WAIT_BUTTON_RELEASE )
3              begin
4                  lcd_rs_reg = lut_out[DATA_WIDTH-1];
5                  lcd_data_reg = lut_out[DATA_WIDTH-2:0];
6                  lcd_en_reg = lcd_en_cntr;
7              end else begin
8                  lcd_rs_reg = lcd_rs_freq;
9                  lcd_data_reg = lcd_data_freq;
10                 lcd_en_reg = lcd_en_freq;
11             end
12             //Each BCD (binary coded decimal) digit needs 4 bits
13             //number extracted from most significant BCD
14             //to least significant BCD (reverse index)
15             bcd_number = bcd_freq[(LAST_BCD_INDEX-(n_index*4))+:BCD_BITS];
16             if( bcd_number >= 4'd9 ) begin
17                 bcd_number = 9;
18             end
19         end

```

Listing 14: Registers multiplexing for FSMs

The core of the process to send data to the LCD is here, this mechanism is present in both FSMs, even if in the following piece of code we are going to analyze the first FSM.

At the start of the system it is necessary to wait more than 15ms as stated on the datasheet of the CFAH1602B-TMC-JP. Each new instruction or character writing starts in the SETUP state, the chip enable is set low and a wait time is scheduled, then in the WRITE_DATA state the enable is set high enabling the sampling of the data to send to the LCD controller, after that the enable is set low again, and a wait for the execution (in the WAIT_EXEC) is needed before going to the next state, the one responsible to select the next character (or instruction) and eventually the page.

```

1      case (state)
2
3          // Wait for more than 15ms at power on
4          POWER_ON: begin
5              lcd_en_cntr <= 0;
6              counter <= counter + 1'b1;
7              if (counter >= wait20ms) begin
8                  state <= SETUP ;
9                  counter <= 0;
10             end
11         end
12
13         // Setup time wait
14         SETUP: begin
15             lcd_en_cntr <= 0;
16             counter <= counter + 1'b1;
17             if (counter >= wait40us) begin
18                 state <= WRITE_DATA;
19                 counter <= 0;
20             end
21         end
22
23         // Enable pulse high
24         WRITE_DATA: begin
25             lcd_en_cntr <= 1;
26             counter <= counter + 1'b1;
27             if (counter >= wait1ms) begin
28                 state <= WAIT_EXEC;
29                 counter <= 0;
30             end
31         end
32
33         // Wait instruction execution
34         WAIT_EXEC: begin
35             lcd_en_cntr <= 0;
36             counter <= counter + 1'b1;
37             if (counter >= wait2ms) begin
38                 char_index <= char_index + 1'b1;
39                 state <= PAGE_SELECT;
40                 counter <= 0;
41             end
42         end
43
44         PAGE_SELECT: begin
45
46             .
47             .
48             .
49
50     endcase

```

Listing 15: Sending data to LCD

Text LUT instance

A LUT is instantiated with the instructions and ASCII coded characters needed to display the first row of the screen, the one related to the name of the selected waveform and the first two opening pages.

This particular LUT is addressable with 2 indexes, one that divides the LUT in pages (`page_index`), for a maximum of 32 pages.

Each page is composed of 64 memory locations, addressable with the index `char_index`.

Each address outputs a 9 bit value which has its MSB related to the register select signal (`lcd_rs`) to send to the LCD controller, and the remaining 8 bits are related to the data bus (`lcd_data`). The resulting dimension of the LUT is:

$$\text{TEXT_LUT_DIM} = 32 \cdot 64 \cdot 9 = 2^{11} \cdot 9 = 18432\text{bits} \simeq 18\text{Kb}$$

```
1 // text LUT instance
2 localparam DATA_WIDTH = 9;
3 localparam ADDR_WIDTH = 6;
4 localparam PAGE_ADDR_WIDTH = 5;
5 reg [ADDR_WIDTH-1:0] char_index = 0;
6 reg [PAGE_ADDR_WIDTH-1:0] page_index = 0;
7 wire [DATA_WIDTH-1:0] lut_out;
8 text_lut #(
9     .DATA_WIDTH(DATA_WIDTH),
10    .ADDR_WIDTH(ADDR_WIDTH),
11    .PAGE_ADDR_WIDTH(PAGE_ADDR_WIDTH)
12 ) text_lut_inst (
13     .addr(char_index),
14     .page(page_index),
15     .clk(clk),
16     .q(lut_out)
17 );
```

Listing 16: LUT instance

First FSM: displaying wave function name

In the PAGE_SELECT state a sub state strategy is used for each page.

The LUT is scanned from the first entry of each page by progressively incrementing char_index (in the WAIT_EXEC state), repeating the states cycle SETUP, WRITE_DATA, WAIT_EXEC.

At each new address, the output of the LUT drives the lcd_rs_reg and lcd_data_reg.

This cycle continues until an @ (hat sign) coded in ASCII is detected, or in case of an error, the end of the page is reached, and an exclusive page signaling the error is displayed.

While the first two pages are only used to display information about the project, thus they are displayed for only 2 seconds and automatically shifted, the page starting with the wave function selection capability needs to implement a button press detection and release before displaying a new page.

```
1  PAGE_SELECT: begin
2
3      case(sub_state)
4
5          // DDFS Tonarini Aliberti
6          PAGE0: begin
7              if (char_index < (2**ADDR_WIDTH)-1) begin
8                  if (lcd_data_reg == 8'h40) begin
9                      char_index <= 0;
10                     page_index <= PAGE1;
11                     sub_state <= PAGE1;
12                     state <= WAIT;
13                 end else begin
14                     state <= SETUP;
15                 end
16             end else begin
17                 char_index <= 0;
18                 sub_state <= PAGE_ERROR;
19             end
20         end
21
22         // Select wave function
23         PAGE1: begin
24             if (char_index < 2**ADDR_WIDTH-1) begin
25                 if (lcd_data_reg == 8'h40) begin
26                     char_index <= 0;
27                     page_index <= PAGE2;
28                     sub_state <= PAGE2;
29                     state <= WAIT;
30                 end else begin
31                     state <= SETUP ;
32                 end
33             end else begin
34                 char_index <= 0;
35                 sub_state <= PAGE_ERROR;
36             end
37         end
38
39         .
40         .
41         .
42
43     endcase
44
45 end
```

Listing 17: Page selection Sub State Machine

The following code shows the states of the first FSM used to wait for a button to be pressed and released when selecting the wave function.

A de-bounce technique is implemented with a 10ms sensing time, and the `page_index` is incremented or decremented based on which of the buttons is pressed.

```
1  WAIT: begin
2      counter <= counter + 1'b1;
3      if (counter >= wait2s) begin
4          state <= SETUP;
5          counter <= 0;
6      end
7
8  end
9
10 WAIT_BUTTON_SHORT: begin
11     if(button_next == 1'b0 || button_prev == 1'b0) begin
12         btn_counter <= btn_counter + 1'b1;
13         if (btn_counter >= wait10ms) begin
14             state <= WAIT_BUTTON_RELEASE ;
15             btn_counter <= 0;
16             if (button_next == 1'b0) begin
17                 btn_pressed <= BTN_NEXT;
18             end else begin
19                 btn_pressed <= BTN_PREV;
20             end
21         end
22     end else begin
23         btn_counter <= 0;
24         state <= WAIT_BUTTON_SHORT;
25     end
26 end
27
28 WAIT_BUTTON_RELEASE: begin
29     if(button_next == 1'b1 && btn_pressed == BTN_NEXT) begin
30         btn_counter <= btn_counter + 1'b1;
31         if (btn_counter >= wait10ms) begin
32             state <= SETUP;
33             btn_counter <= 0;
34             if (page_index >= PAGE_FIRST && page_index < PAGE_LAST)
35                 begin
36                     page_index <= page_index + 1'b1;
37                 end else if ( page_index == PAGE_LAST ) begin
38                     page_index <= PAGE_FIRST;
39                 end
40             end
41         end else if (button_prev == 1'b1 && btn_pressed == BTN_PREV) begin
42             btn_counter <= btn_counter + 1'b1;
43             if (btn_counter >= wait10ms) begin
44                 state <= SETUP;
45                 btn_counter <= 0;
46                 if (page_index > PAGE_FIRST && page_index <= PAGE_LAST)
47                     begin
48                         page_index <= page_index - 1'b1;
49                     end else if ( page_index == PAGE_FIRST ) begin
50                         page_index <= PAGE_LAST;
51                     end
52                 end
53             end else begin
54                 state <= WAIT_BUTTON_RELEASE;
55                 btn_counter <= 0;
56             end
57         end
58     end
59 end
```

Listing 18: Button press and release

Second FSM: displaying wave function frequency

This FSM is used to show the frequency of the selected wave function on the second row of the LCD.

It starts cycling through the states when the first FSM is waiting for a button to be pressed or released.

The DIGIT_SELECT state provides the ASCII code, calculated from the BCD number received as input, to the lcd_data_freq register to display each digit on the screen.

When all digits are correctly displayed the HZ_DISPLAY state print on the screen " Hz" at the end of the frequency value.

It is important to note that, again, like the first FSM, to display each character a cycle of the writing states needs to be executed (SETUP, WRITE_DATA, WAIT_EXEC).

```
1  DIGIT_SELECT: begin
2      lcd_rs_freq <= 1'b1;
3      lcd_data_freq <= bcd_number + 8'h30;
4      n_index <= n_index + 1'b1;
5      if ( n_index >= BCD_DIGITS ) begin
6          state_f <= HZ_DISPLAY;
7      end else begin
8          state_f <= SETUP;
9      end
10 end
11
12 HZ_DISPLAY: begin
13     case(sub_state_f)
14     0: begin
15         lcd_data_freq <= 8'h20;
16         sub_state_f <= sub_state_f + 1'b1;
17         state_f <= SETUP;
18     end
19     1: begin
20         lcd_data_freq <= 8'h48;
21         sub_state_f <= sub_state_f + 1'b1;
22         state_f <= SETUP;
23     end
24     2: begin
25         lcd_data_freq <= 8'h7A;
26         sub_state_f <= sub_state_f + 1'b1;
27         state_f <= SETUP;
28     end
29     3: begin
30         sub_state_f <= 0;
31         state_f <= CHECK_AND_WAIT;
32     end
33 endcase
34 end
```

Listing 19: Digit selection and Hz display

2.4 Clock distribution

The project is composed of different modules synchronized with a common clock source.

The main clock lines are the external 50MHz clock and the PLL multiplied 70MHz clock.

These two clocks have the same phase because the internal FPGA PLL has been configured to make this possible.

The three synchronized main modules are

- keypad_freq_sel_module
- ddfs
- lcd_driver

The keypad_freq_sel_module has an internal clock enable to reduce the throughput (to scan the keypad more slowly) and to communicate with the other modules using the same clock source.

The ddfs module has an internal frequency clock divider, programmable through the external pins of the module.

The problem with using an internal frequency divider is that the divided clock is shifted by a time t_{co} (clock-to-output time) of the flip-flop that generates it, and this can lead to setup and hold time violations.

The output of the module goes only to the DAC, and the DAC clock is connected to the output of the frequency divider; the problem is that the `f_w` and the `mirror_x`, `mirror_y` inputs are sampled with the divided clock of the frequency divider.

The delay is not a problem, because the `f_w` and the `mirror_x`, `mirror_y` inputs remain constant for all the time that the output waves are shown, which is usually seconds or minutes; therefore, even if for some clock cycles the data is not sampled correctly, after some time the situation is fixed automatically.

The main problem is metastability due to hold or setup time violations.

To analyze this system, the output of the frequency divider is considered by the timing analyzer as a fixed clock with the maximum clock frequency (worst-case analysis), and it verifies whether the setup and hold slack is greater than zero.

The issue is that the timing analyzer considers the signal to be in phase with the original clock, which is not the case.

Therefore, to protect the system from metastability, the use of a latch chain to protect the `f_w` and the `mirror_x`, `mirror_y` inputs is suggested.

Other solutions has been analyzed in section 5.

3 Test-plan

3.1 Functional verification

The functional verification of a verilog module is the process of checking that the module behaves exactly as intended according to its specification.

It is typically done through simulation using testbenches that apply input signals and observe outputs over time.

The goal is to find logical and design errors before synthesis.

3.1.1 DDFS module

The test-plan for the DDFS module consist on checking every type of output waveform with different `f_w` and `freq_cntrl` values.

Testbench

The testbench applies every combination of the pins `sin`, `triang`, `mirror_x`, `mirror_y` for 50000 clock cycles with `f_w = 0` and `freq_cntrl = 0`.

Then, the value of `f_w` is changed to observe how the frequency and the accuracy of the output wave vary with this parameter.

```
1      //testing every combination of input signals
2      for(i=0; i<4'b1111;i=i+1) begin
3          #(CLOCK_PERIOD*50000)
4          {mirror_x, mirror_y, sin, triang} = i;
5      end
6
7      //apply sine wave
8      #(CLOCK_PERIOD*50000)
9      {sin, triang, mirror_x, mirror_y} = 4'b1111;    //testing sin priority
10
11     //changing frequency word
12     for(i=0; i<7'b1111111;i=i+10) begin
13         #(CLOCK_PERIOD*50000)
14         fw = i;
15     end
```

Listing 20: DDFS testbench

The `freq_cntrl` parameter variation has been tested in the frequency divider testbench.

In the frequency divider testbench, the clock output of the frequency divider module has been analyzed as the `freq_cntrl` parameter value is varied.

Wave analysis

To analyze the output waveform of the DDFS module, it has been represented both as an unsigned value and in analog form.

This allows the digital numbers to be visualized as an analog waveform.

The first thing to observe from the testbench is the output waveform for every combination of the input values (while keeping the same value of f_w).

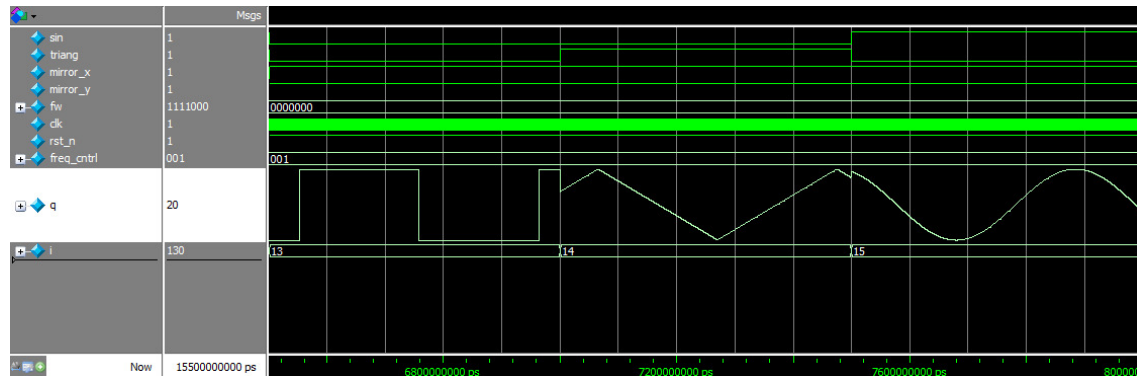


Figure 9: DDFS testbench output wave

In Figure 9 are shown the three main waveform with both mirroring effect enabled.

The second thing that can be analyzed from the testbench is the variation of the output frequency corresponding to a swing of f_w .

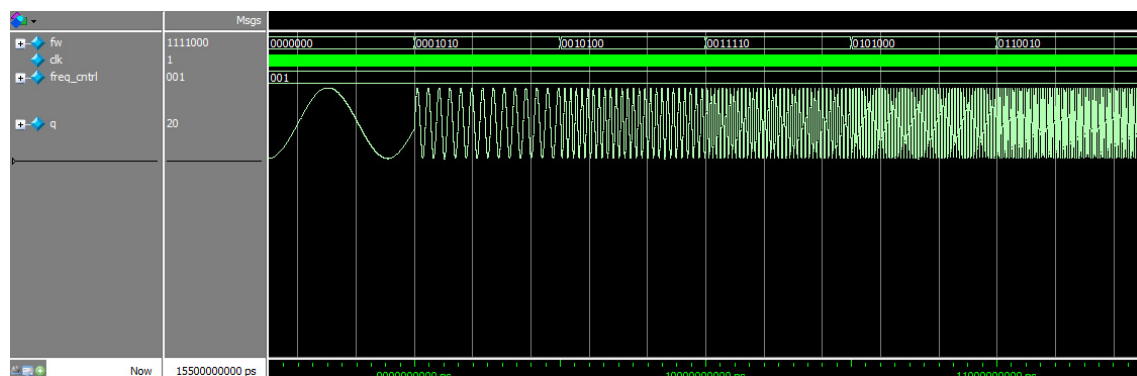


Figure 10: DDFS sine-wave frequency swing

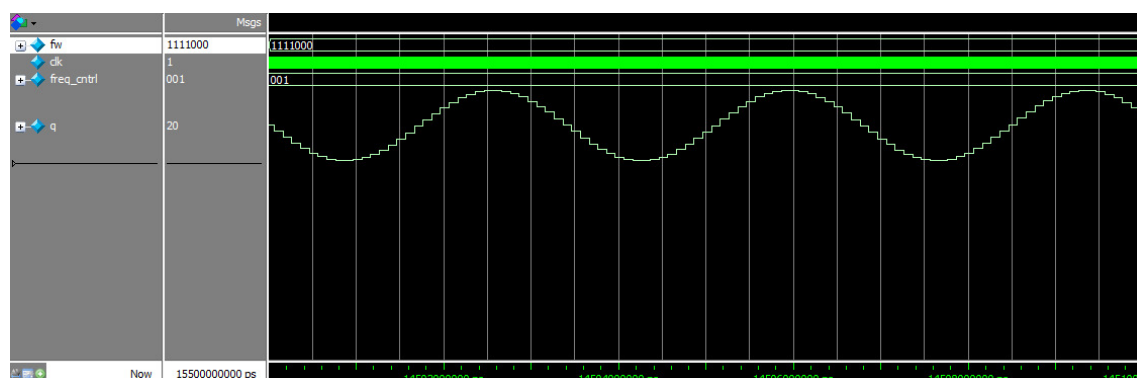


Figure 11: DDFS sine-wave with biggest value of f_w

For the frequency divider, only the output frequencies $\frac{f_{clk}}{2}$, $\frac{f_{clk}}{10}$, $\frac{f_{clk}}{100}$, and $\frac{f_{clk}}{1000}$ were tested, since the other divisions are too large to be simulated.

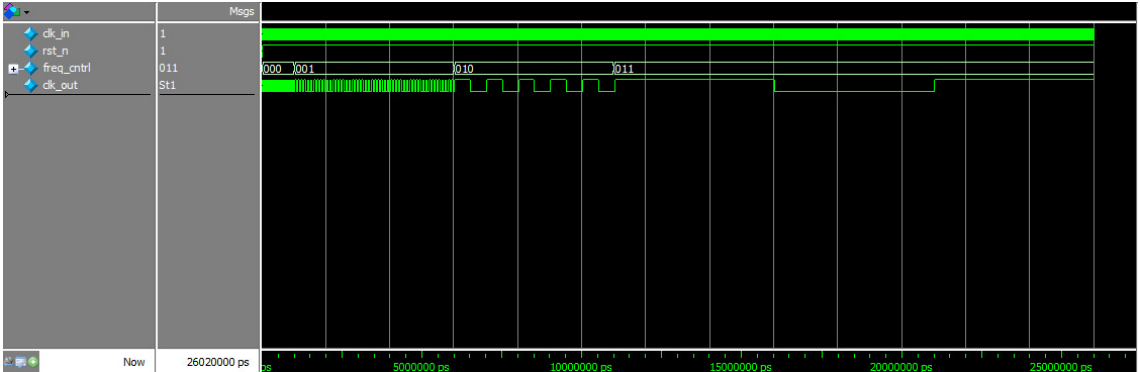


Figure 12: DDFS divided frequency from frequency divider

3.1.2 Keypad frequency select module

The test procedure for this module is divided into different parts: the test of the keypad module, the test of the frequency converter module, and the test of the modules connected together.

To simulate the behavior of the 4×4 matrix keypad (used for the keypad module and for the keypad frequency selection module), a dedicated module has been implemented.

This module is controlled via an input bus and simulates the electrical connection between the corresponding columns and rows.

```
1  module keypad_4x4
2  (
3      // ---- KEYPAD PINS ---- //
4      input  [3:0] column,
5      output reg [3:0] row,
6
7      // ---- BUTTON PINS ---- //
8      input  [3:0] button,
9      input  button_pressed
10 );
11
12     integer i;
13     always @(*) begin
14         //assign button value
15         if(button_pressed == 1)
16             for(i=0; i<4; i=i+1) begin
17                 if(i == button/4)
18                     row[i] = column[button%4];
19                 else
20                     row[i] = 1;
21             end
22         else
23             row = 4'b1111;
24     end
25
26 endmodule
```

Listing 21: 4x4 keypad simulator module

Testbench

To test the keypad module, the press of a different keypad button was simulated every 100ns, and the values of the out_keys and data_valid outputs were read.

```
1  //none button pressed
2  button_pressed = 0;
3
4  for(i=0; i<=4'b1111; i=i+1) begin
5      //press a button
6      #100
7      button_pressed = 1;
8      button = i;
9  end
10
11 //none button pressed
12 #100 button_pressed = 0;
```

Listing 22: Keypad module testbench

To test the DDFS frequency converter module, the value of the module input frequency was increased from 0 to MAX_FREQ, which was set to 1MHz, with a CLK_FREQ of 70MHz. Due to the large range of frequency values, the increment step was set to 100.

```

1  for(i=0; i<=MAX_FREQ; i=i+100) begin
2      #10 freq = i;
3  end

```

Listing 23: DDFS converter module testbench

To test the accuracy of the module with different clock scaling factors, the output waveform frequency was also calculated using the f_w and freq_cntrl values processed by the module. Then, the delta_freq was calculated as the difference between the ideal frequency (the input of the DDFS converter module) and the real frequency (calculated from the output of the DDFS converter module).

```

1  reg [63:0] divided_clock;
2  always @(*) begin
3      case(freq_control)
4
5          //select the correct divided_clock output
6          3'd0: divided_clock = CLK_FREQ/2;
7          3'd1: divided_clock = CLK_FREQ/10;
8          3'd2: divided_clock = CLK_FREQ/100;
9          3'd3: divided_clock = CLK_FREQ/1000;
10         3'd4: divided_clock = CLK_FREQ/10000;
11         3'd5: divided_clock = CLK_FREQ/100000;
12         3'd6: divided_clock = CLK_FREQ/1000000;
13
14         default: begin
15             //error stop simulation
16             $error("ERROR: frequency divider value out of range");
17             $stop;
18         end
19     endcase
20 end
21
22 assign @(*) begin
23
24     if(mirror_x == 0 && mirror_y == 0) begin
25         freq_calculated = ((fw+1)*divided_clock)/1024;
26     end
27     else if(mirror_x == 0 || mirror_y == 0) begin
28         freq_calculated = 2*((fw+1)*divided_clock)/1024;
29     end
30     else begin
31         freq_calculated = 4*((fw+1)*divided_clock)/1024;
32     end
33 end
34
35 end
36
37 assign delta_freq = freq - freq_calculated;

```

Listing 24: Delta frequency calculation

Wave analysis

From the keypad module testbench, it is possible to observe the column enabling cycle and the row reading cycle.

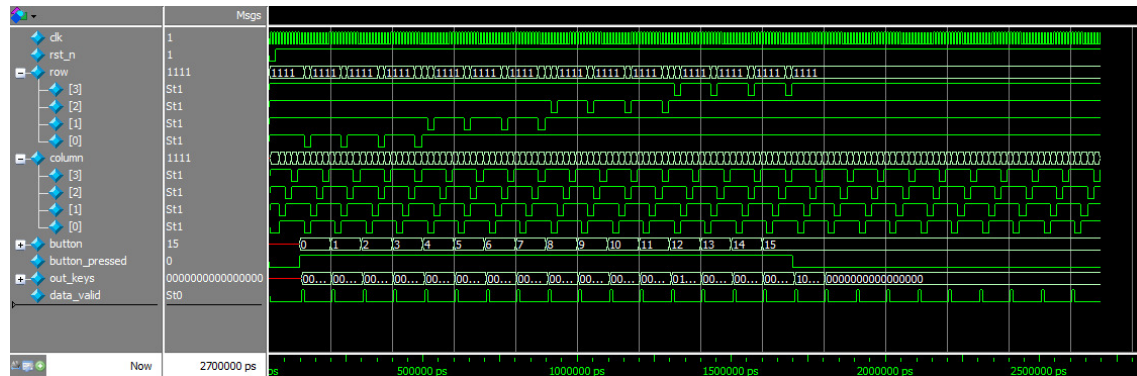


Figure 13: Keypad module waveform of scan cycles

Enabling a column and reading the corresponding rows takes two clock cycles.

The output data (rows and column of the button pressed) is sampled and stored into a buffer and at the end of the scanning cycle the value of the buffer is stored into the output vector.

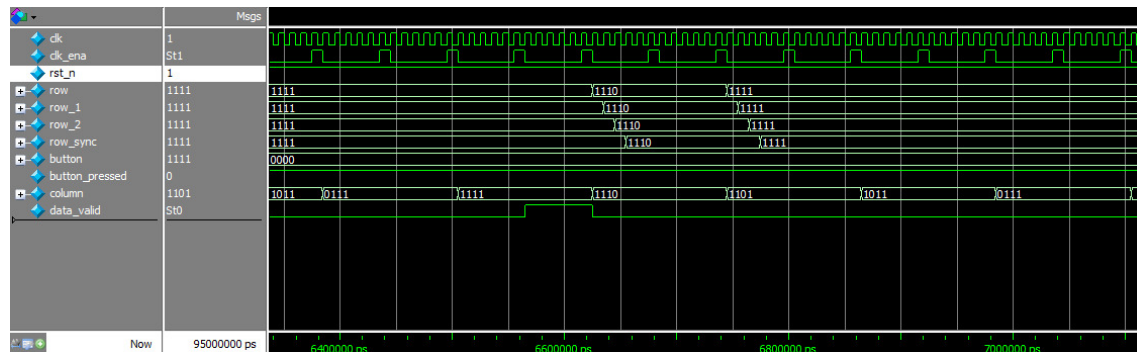


Figure 14: Sampling data of keypad module

In Figure 14 we can see that the row inputs pass through three flip-flops to reduce metastability problems, but this does not cause trouble, because the clock enable rises after row_sync has stabilized.

By analyzing the DDFS converter module testbench, we can observe the variation of `delta_freq` for a sweep of the input frequency from 0 Hz to 1 MHz, and calculate the maximum absolute (and relative) frequency error for each frequency division (from the frequency divider).

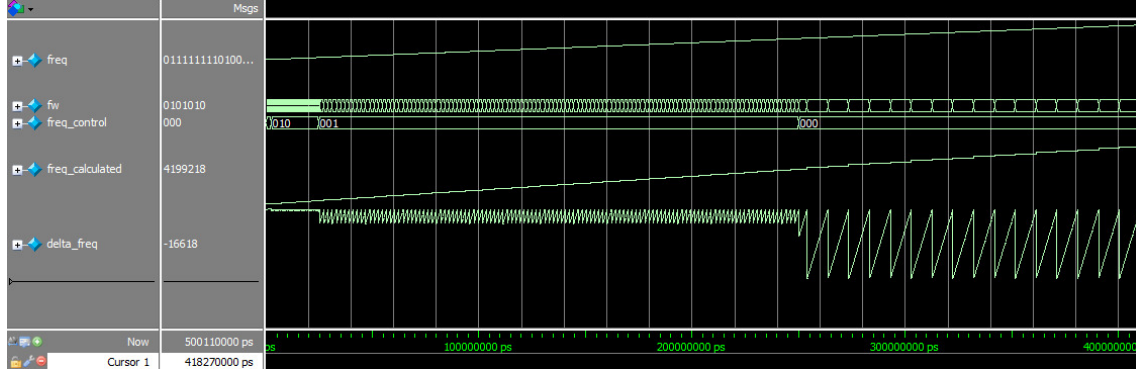


Figure 15: Testbench output of DDFS converter module

As we can see in Figure 15, the absolute value of `delta_freq` is lower than a value that depends on the frequency control calculated by the module.

The `delta_freq` value is equal to the absolute frequency error.

- `freq_control = 000`

$$|\epsilon_a| = |f_{in} - f_{measured}| < 33000Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| < \left| \frac{\max(\epsilon_a)}{\min(f_{in})} \right| \simeq 3.8\%$$
- `freq_control = 001`

$$|\epsilon_a| = |f_{in} - f_{measured}| < 6500Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| < \left| \frac{\max(\epsilon_a)}{\min(f_{in})} \right| \simeq 7.2\%$$
- `freq_control = 010`

$$|\epsilon_a| = |f_{in} - f_{measured}| < 650Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| < \left| \frac{\max(\epsilon_a)}{\min(f_{in})} \right| \simeq 7.0\%$$
- `freq_control = 011`

$$|\epsilon_a| = |f_{in} - f_{measured}| < 65Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| < \left| \frac{\max(\epsilon_a)}{\min(f_{in})} \right| \simeq 7.1\%$$
- `freq_control = 100`

$$|\epsilon_a| = |f_{in} - f_{measured}| < 6Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| < \left| \frac{\max(\epsilon_a)}{\min(f_{in})} \right| \simeq 6.7\%$$
- `freq_control = 101`

$$|\epsilon_a| = |f_{in} - f_{measured}| < 1Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| < \left| \frac{\max(\epsilon_a)}{\min(f_{in})} \right| \simeq 0\%$$
- `freq_control = 110`

$$|\epsilon_a| = |f_{in} - f_{measured}| = 0Hz \Rightarrow |\epsilon_r| = \left| \frac{f_{in} - f_{measured}}{f_{in}} \right| = 0\%$$

3.1.3 LCD module

We want to test the correct states change of the two FSM of the LCD module.

Testbench

Only for simulation purposes the delays needed to drive the LCD are reduced to a fixed value of 40 ns, this way it is simpler to test the module with Modelsim. To solve this scope a testing Verilog file is created (lcd_driver_test.v)

```
1      initial begin
2          reset_n = 1'b0;
3          button_next = 1'b1;
4          button_prev = 1'b1;
5          bcd_freq = 28'hFFFFFFF;
6
7          #(CLK_PERIOD*2)
8          reset_n = 1'b1;
9
10         #(CLK_PERIOD*2000)
11         button_next = 1'b0;
12
13         #(CLK_PERIOD*10)
14         button_next = 1'b1;
15
16         #(CLK_PERIOD*1000)
17         button_next = 1'b0;
18
19         #(CLK_PERIOD*10)
20         button_next = 1'b1;
21
22         #(CLK_PERIOD*1000)
23         button_prev = 1'b0;
24
25         #(CLK_PERIOD*10)
26         button_prev = 1'b1;
27
28         #(CLK_PERIOD*1000)
29         // Finish simulation
30         $stop;
31     end
```

Listing 25: LCD module testbench

Wave analysis

The simulation starts with a reset and after that the POWER_ON state is enabled, followed by the first writing cycle to the LCD. As we can see, at first, in the lcd_data register is stored the first instruction present in the LUT to send to the LCD (hex38, initializing the function set), then when the first writing cycle is completed the next instruction is read from the LUT and loaded in the lcd_data register.

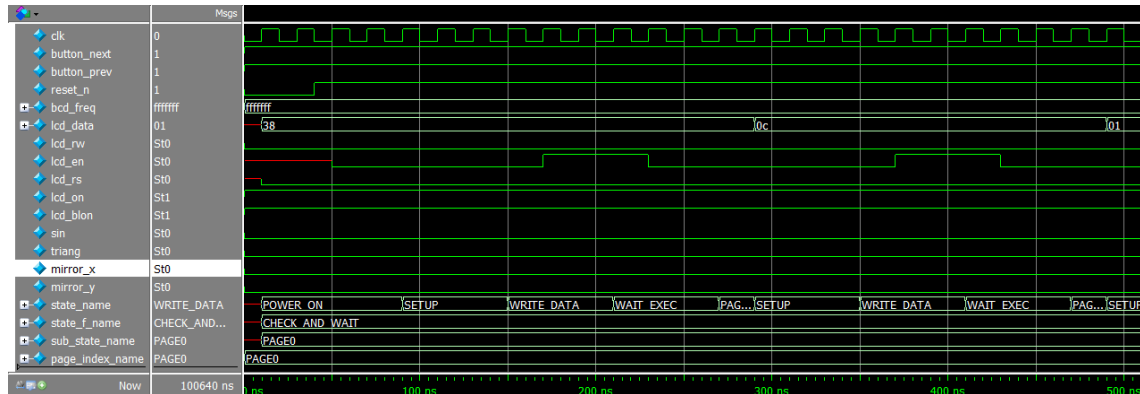


Figure 16: LCD power on

After sending the first instruction to initialize the LCD we can start to send ASCII characters by setting the register select flag to 1. In Figure 17 we see the ASCII coded characters that are written to the DDRAM of the LCD, each in sequence with a writing cycle.

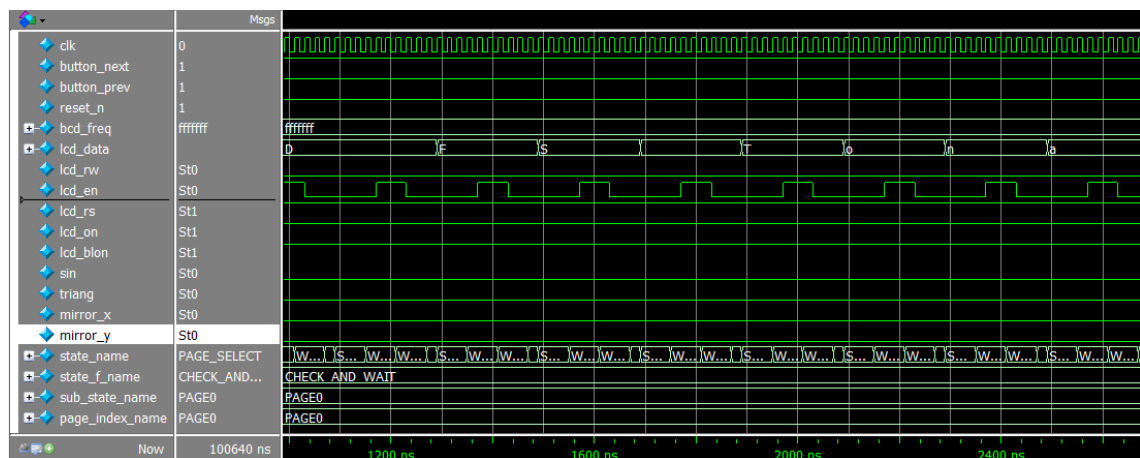


Figure 17: Displaying characters

When a page is completely displayed the first FSM enters the WAIT_BUTTON_SHORT state, waiting for the button to be pressed and released. At this point the second FSM is activated, displaying the frequency digits received as BCD input vector in the second row of the display (Figure 18).

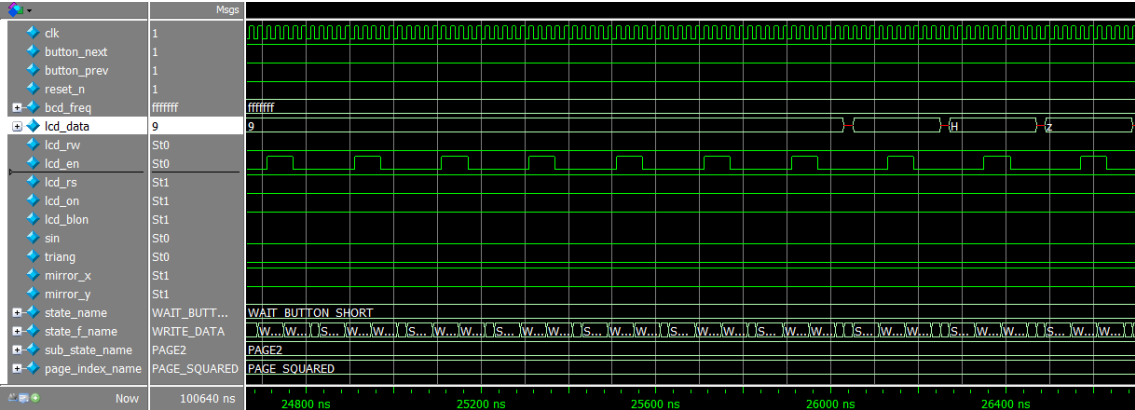


Figure 18: Displaying frequency digits

In the example in Figure 19 the button is pressed to change the wave function to display, changing from squared wave to triangular wave, the output `triang` is enabled to signal the new waveform to the DDFS module. Since a new page with new characters needs to be displayed, a new writing cycle of the first FSM starts, the second FSM starts waiting until the first FSM has completed the job (entering the WAIT_BUTTON_SHORT state again).

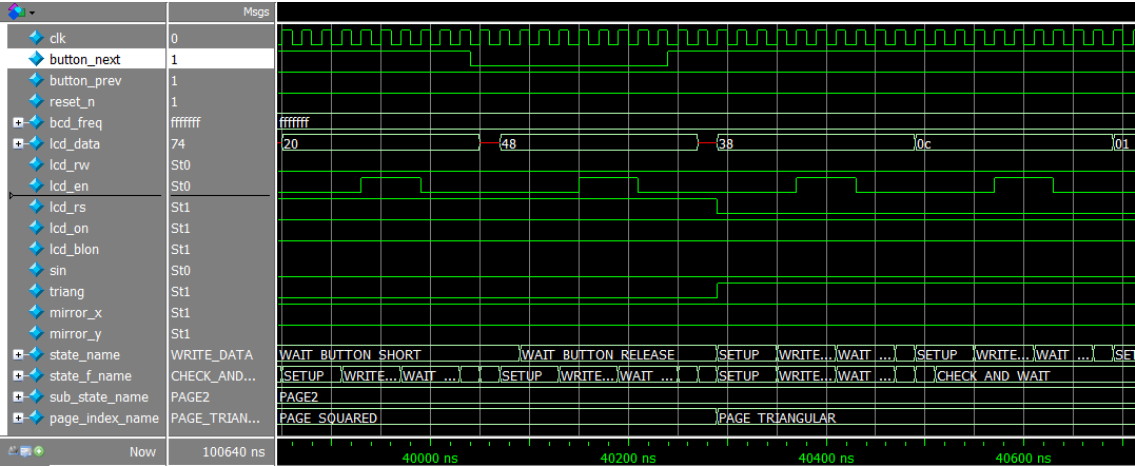


Figure 19: Changing wave function

3.2 On-board module testing

To verify that each module works properly on the board, each module has been tested individually on the board, using the keys, buttons, and LEDs present on the **Altera DE2 development board**.

3.2.1 DDFS module

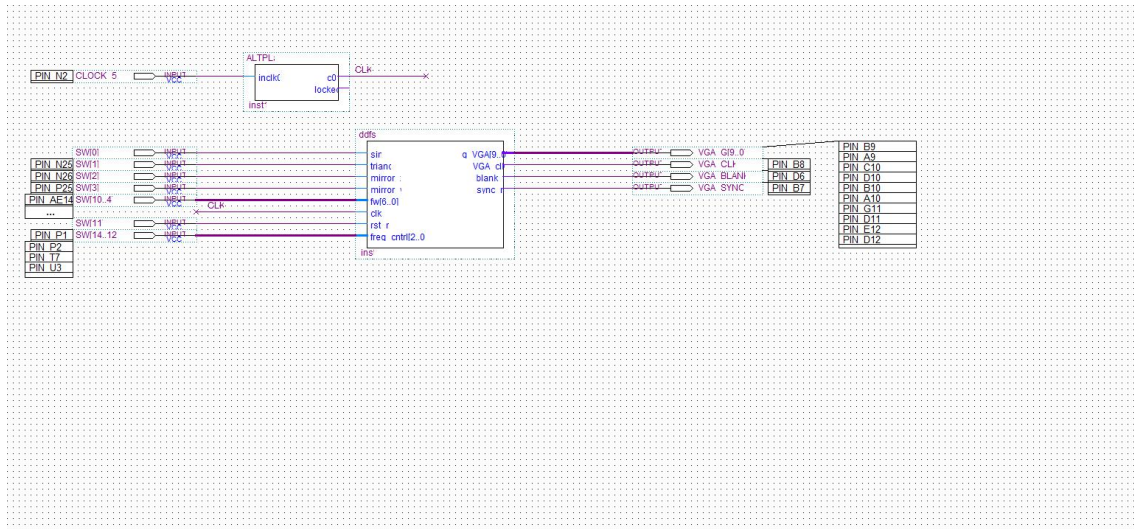


Figure 20: DDFS onboard test schematic

As we can see from Figure 20, the module inputs have been connected to the keys on the board.

The output has been connected to the VGA DAC inputs.

The input 50MHz clock source from the board passes through the internal PLL of the FPGA and becomes a 70MHz clock signal.

The oscilloscope can be connected to the green channel of the VGA port to monitor the analog output signal of the waveform generator and verify that the output matches the module's input.

3.2.2 Keypad frequency select module

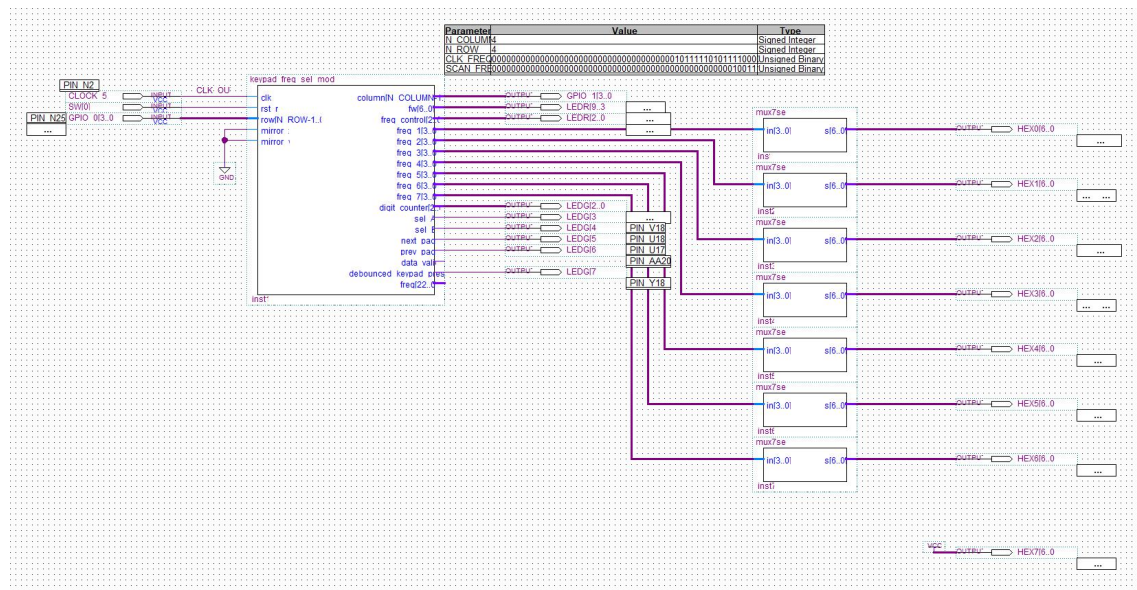


Figure 21: keypad frequency select onboard test schematic

The pins of the keypad have been connected to the GPIO of the board, and the input pins must be configured as input pull-up.

The output of the module has been connected, through a BCD-to-7-segment display converter module, to the 7-segment displays on the board, to control the digit shift and the selection of the output signal.

The clock source comes from the oscillator on the board but the module work with a clock enable signal, to match the frequency scan value (an external parameter).

The other control pins for the DDFS module are connected to the remaining LEDs on the board.

3.2.3 LCD module

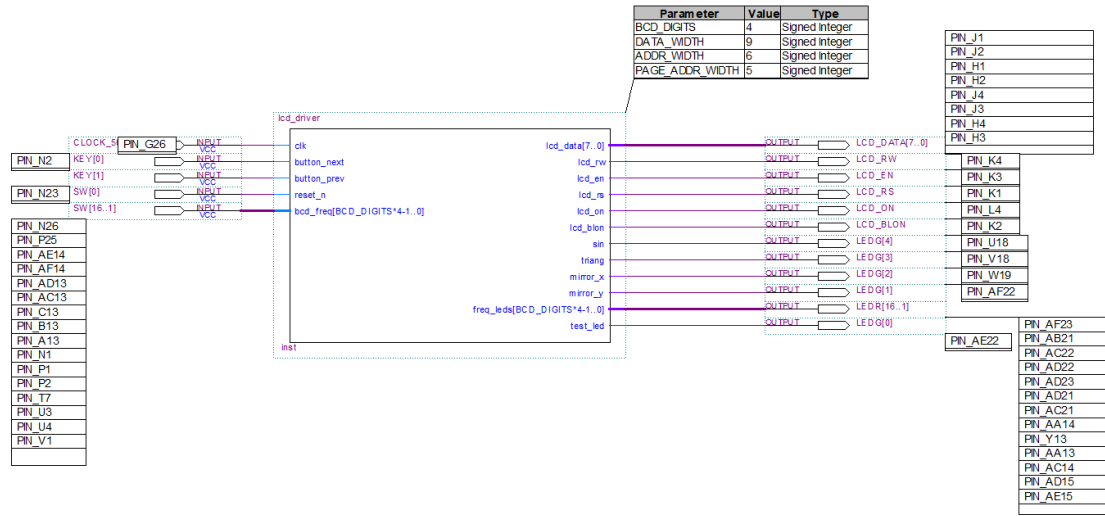


Figure 22: LCD module onboard test schematic

In the testing phase of the LCD module it has been useful to connect the inputs of the `bcd_freq` vector to 16 switches of the board to display at least 4 variable digits (4 switches for each BCD digit), the button for the next and previous page were connected to two physical buttons of the board, and LEDs were used to show the outputs (`sin`, `triang`, `mirror_x`, `mirror_y`) and to test the buttons and switch activation (`test_led`, `freq_leds`).

3.2.4 Physical test of the entire design

In this section some examples and capabilities of the final design are shown.

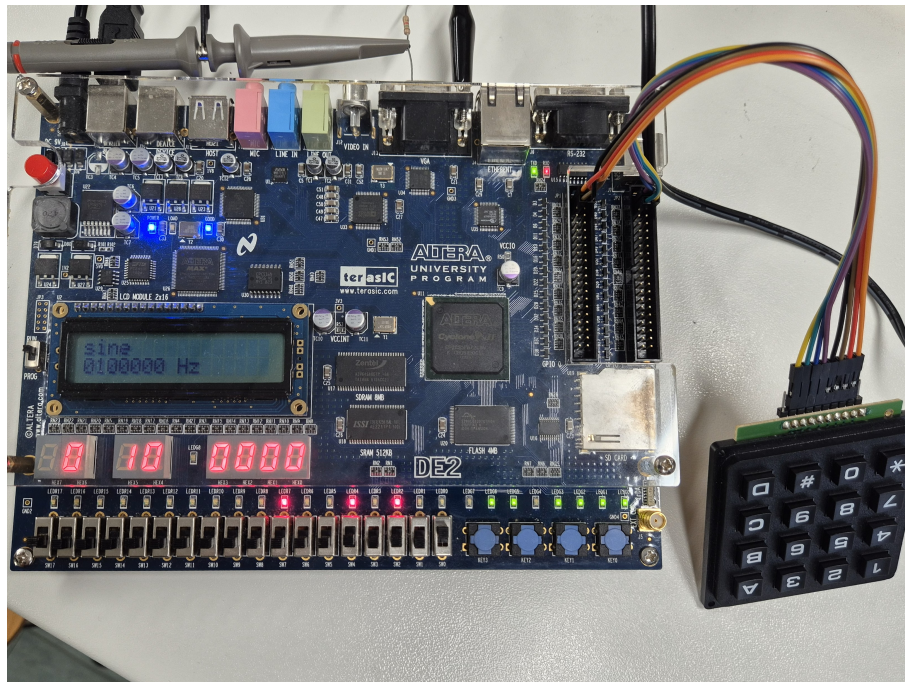


Figure 23: Selecting the sine wave with a 100kHz frequency

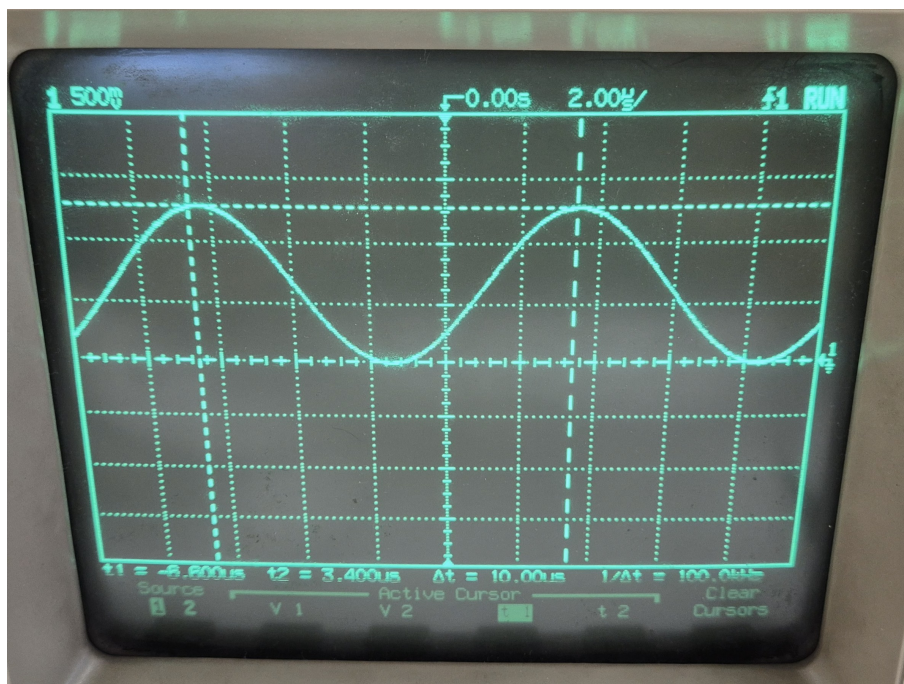


Figure 24: Sine wave, oscilloscope view

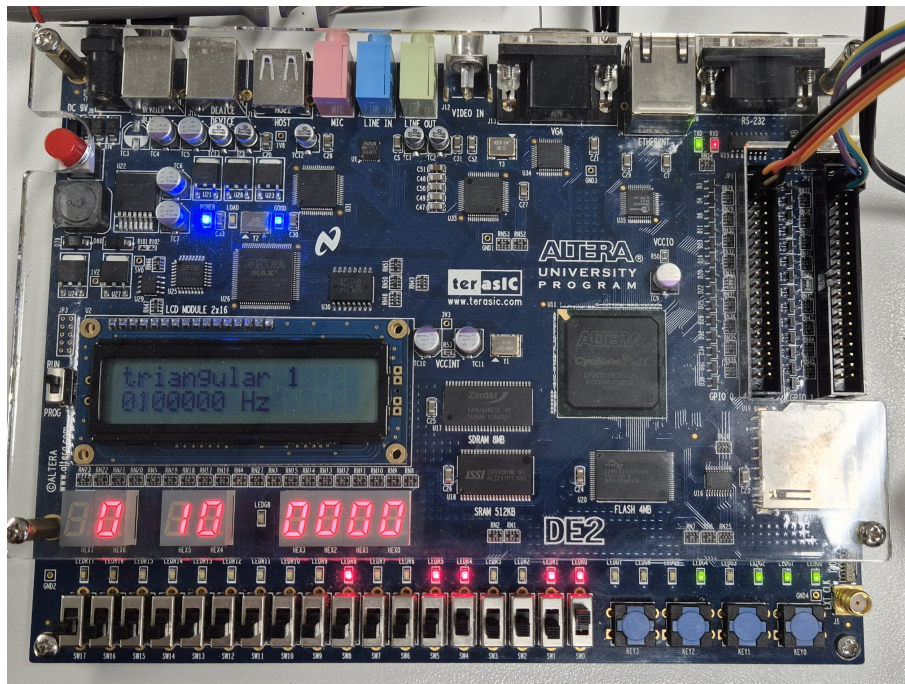


Figure 25: Selecting the sawtooth wave with a 100kHz frequency

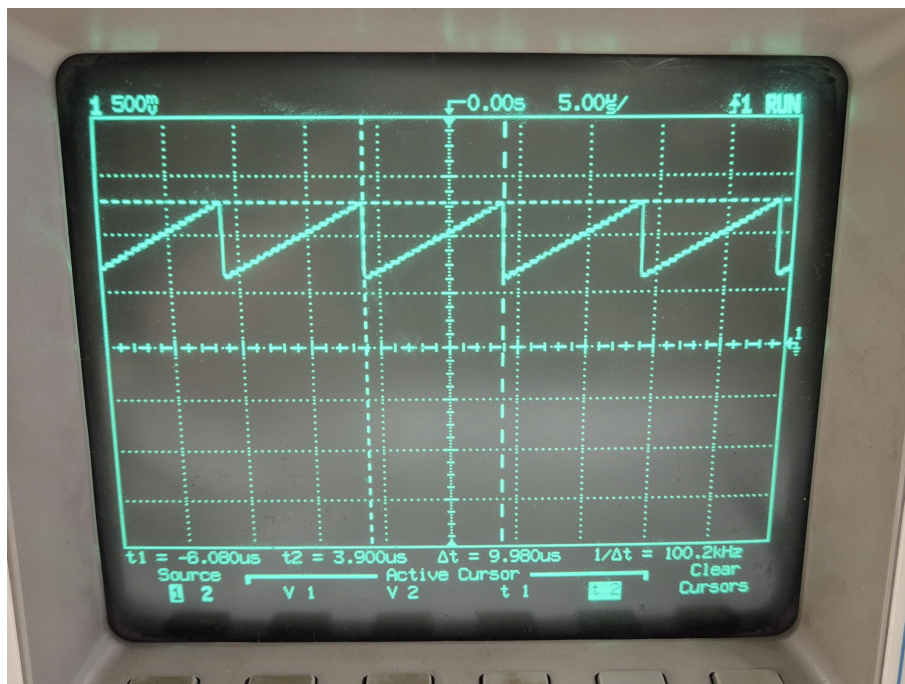


Figure 26: Sawtooth wave, oscilloscope view

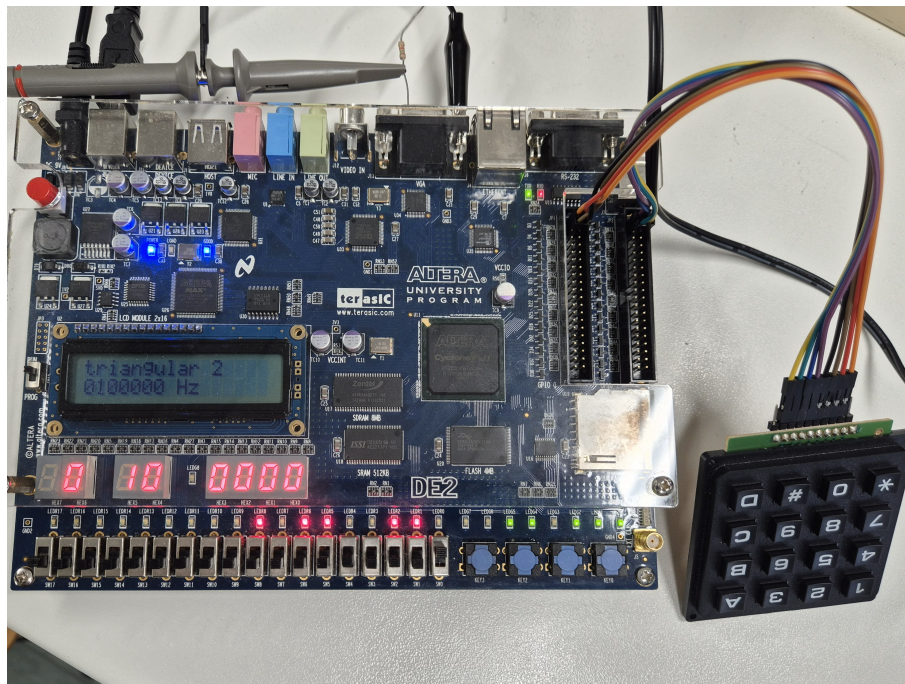


Figure 27: Quarter triangular x mirrored with a 100kHz frequency

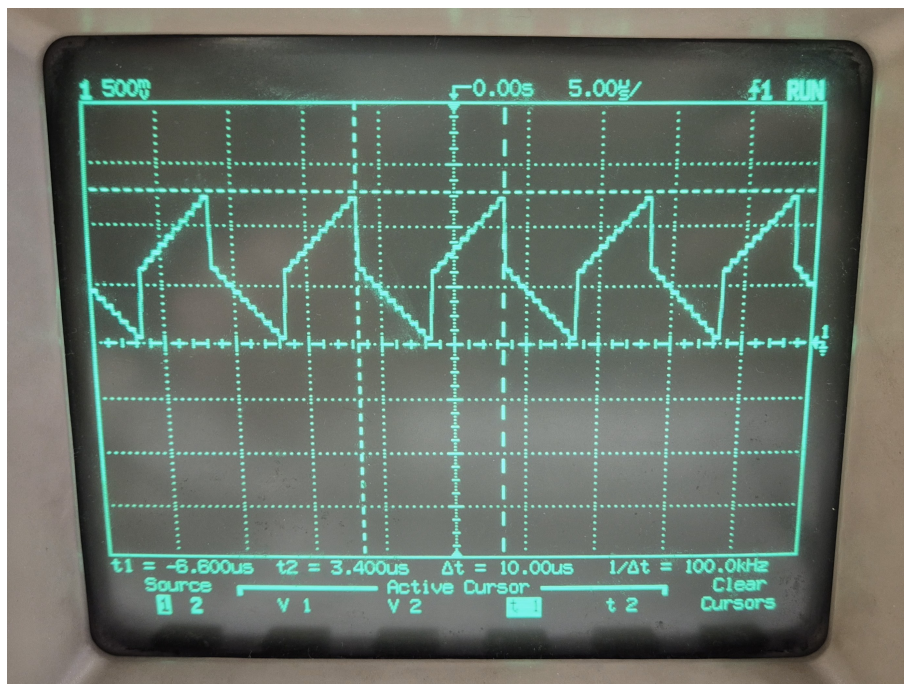


Figure 28: Quarter triangular x mirrored, oscilloscope view

Another test that was conducted verified that the signal from the keypad module becomes increasingly difficult to read as the scan rate increases, until it becomes impossible to read at a clock rate of 50MHz .

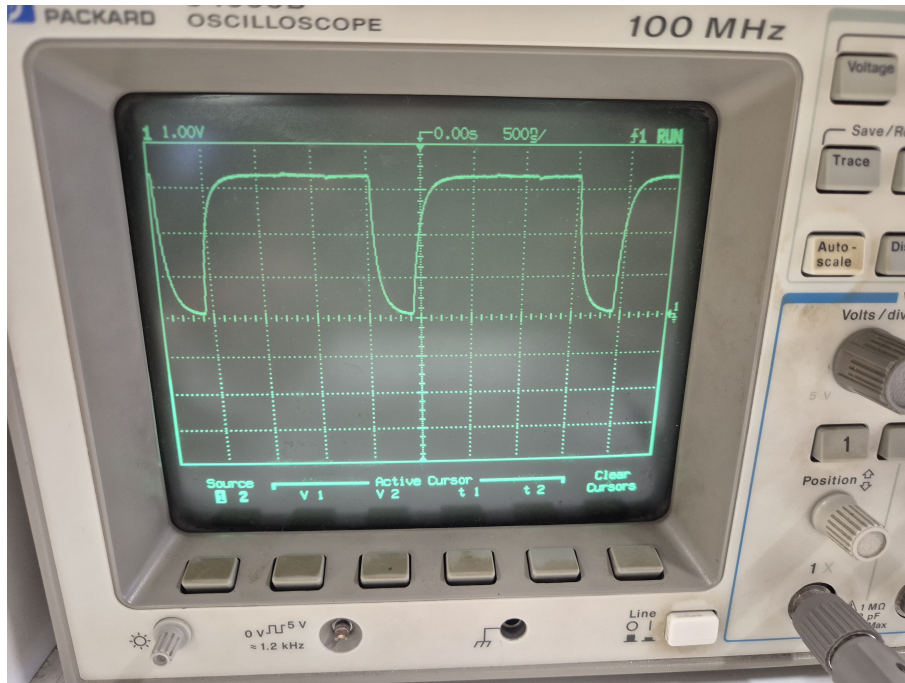


Figure 29: Signal read from the keypad with a scan frequency of 5MHz

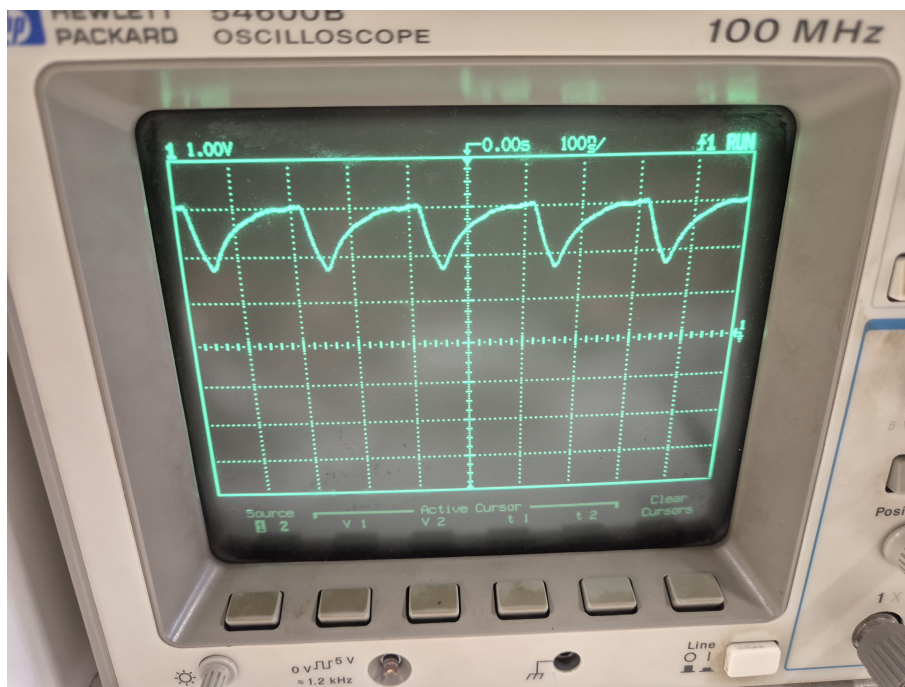


Figure 30: Signal read from the keypad with a scan frequency of 50MHz

4 Implementation on Quartus

4.1 Warnings

The synthesis of the project on the Altera DE2 development board generates some warnings that are safe for the project, but it is important to analyze them to ensure the correct operation of the device.

The analysis of the warnings is separated for each module and for the main project that connects them together.

4.1.1 LCD module

The LCD module generate a lot of warning due to truncation of signals.

The first truncation warnings are as follows.

Warning

Warning (10230): Verilog HDL assignment warning at lcd_driver.v(*):
truncated value with size 64 to match size of target (32)

They are generated for the `localparam` inside the `lcd_driver` module, which contains the values of clock cycles to wait for a specific temporal delay.

```
1      localparam [31:0] wait40us = (CLK_FREQ/1000000)*40;  
2      localparam [31:0] wait1ms = (CLK_FREQ/1000)*1;  
3      localparam [31:0] wait2ms = (CLK_FREQ/1000)*2;  
4      localparam [31:0] wait10ms = (CLK_FREQ/1000)*10;  
5      localparam [31:0] wait20ms = (CLK_FREQ/1000)*20;  
6      localparam [31:0] wait100ms = (CLK_FREQ/1000)*100;  
7      localparam [31:0] wait2s = CLK_FREQ*2;
```

The operation on the right is performed automatically at 64bits .

The other truncation warnings refer to the initialization file for the LUT.

Warning

Warning (10230): Verilog HDL assignment warning at text_lut_init.txt(*):
truncated value with size 12 to match size of target (9)

The instantiated rom has 9bit elements but is initialized with a hexadecimal file; for this reason, the first three bits are truncated, and the memory initialization file generated contains 9bit binary elements.

To allow, in the future, to add other page on the LCD, some memory location of the rom are left uninitialized, and this generate another warning.

Warning

Warning (10850): Verilog HDL warning at text_lut.v(16):
number of words (437) in memory file does not match the number of elements in the
address range [0:2047]

The instantiated rom memory is a read-only memory, so all the pins needed to write the memory are fixed at zero, generating the following warnings.

Warning

Warning (10030): Net "rom.data_a" at text_lut.v(12) has no driver or initial value, using a default initial value '0'
Warning (10030): Net "rom.waddr_a" at text_lut.v(12) has no driver or initial value, using a default initial value '0'
Warning (10030): Net "rom.we_a" at text_lut.v(12) has no driver or initial value, using a default initial value '0'

4.1.2 DDFS module

The DDFS module generates only truncation warnings due to the operations that are performed by default with signed integers at *64bits*.

Warning

Warning (10230): Verilog HDL assignment warning at ddfs.v(*):
truncated value with size 64 to match size of target (12)
Warning (10230): Verilog HDL assignment warning at ddfs.v(*):
truncated value with size 64 to match size of target (10)

The last warnings of this module are referred to the rom instance and are the same of the LCD module.

Warning

Warning (10030): Net "rom.data_a" at sin_lut.v(17) has no driver or initial value, using a default initial value '0'
Warning (10030): Net "rom.waddr_a" at sin_lut.v(17) has no driver or initial value, using a default initial value '0'
Warning (10030): Net "rom.we_a" at sin_lut.v(17) has no driver or initial value, using a default initial value '0'

4.1.3 Keypad frequency module

This module produce only truncation warnings due to the operations that are performed by default with signed integers at *64bits*.

Warning

Warning (10230): Verilog HDL assignment warning at keypad_freq_sel_module.v(49):
truncated value with size 64 to match size of target (32)

Other truncation warnings are generated from the DDFS frequency converter module

Warning

Warning (10230): Verilog HDL assignment warning at DDFS_frequency_converter.v(*):
truncated value with size 65 to match size of target (7)

This warning come from the following part of code

```
1      fw_1000000 <= (mult_1000000*K)>>54;  
2      fw_100000 <= (mult_100000*K)>>54;  
3      fw_10000 <= (mult_10000*K)>>54;  
4      fw_1000 <= (mult_1000*K)>>54;  
5      fw_100 <= (mult_100*K)>>54;  
6      fw_10 <= (mult_10*K)>>54;  
7      fw_2 <= (mult_2*K)>>54;
```

where `mult_*` contains the value of the frequency multiplied by powers of 10.

The value is shifted 54 times to the right, which is equivalent to dividing the value by 2^{54} .

This value can be stored in *7bits*, but the multiplied frequency needs *65bits* to be represented, for this reason Quartus report a warning.

4.1.4 Other warnings

The other warning are related to the connection of the GPIOs and the instance or the optimization of the logic block inside the FPGA.

The optimization warning are as follows

Warning

Warning (14284): Synthesized away the following node(s):

↳ Warning (14285): Synthesized away the following RAM node(s):

↳ Warning (14320): Synthesized away node "ddfs:inst_ddfs_mod ... ram_block1a0"

The meaning of the warning is that an auto generated block has been removed due to synthesis optimization. The reported example refers to a RAM block instantiated inside the `ddfs` module, but there are other similar warnings referring to DSP multiplier blocks instantiated inside the `keypad_freq_sel_module`.

The GPIOs warning instead are different, the first type of GPIO warning are like follows

Warning

Warning (13024): Output pins are stuck at VCC or GND

↳ Warning (13410): Pin "LCD_RW" is stuck at GND

↳ Warning (13410): Pin "LCD_ON" is stuck at VCC

↳ Warning (13410): Pin "LCD_BLON" is stuck at VCC

↳ Warning (13410): Pin "VGA_BLANK" is stuck at VCC

↳ Warning (13410): Pin "VGA_SYNC" is stuck at VCC

↳ Warning (13410): Pin "HEX7[*]" is stuck at VCC

This warning is normal for this design because some output pins of the FPGA are connected to the `blank` and `synch` pins of the VGA DAC, which are not used and are always disabled.

Other disabled pins are the `blon`, `rw`, and `on` pins of the LCD, used to set it in write mode and to keep the backlight always on.

The last disabled output pins are the seven-segment pins of the last seven-segment display, which are turned off because the display is not used.

There are two other types of GPIOs warning, because for the design has been used a `.qsf` file to automatically configure the output pins, but some of the pin are not used in the project, for this reason Quartus report them

Warning

Warning (15705): Ignored locations or region assignments to the following nodes

↳ Warning (15706): Node " ... " is assigned to location or region, but does not exist in design

furthermore some of the output pins have no defined the output capacitance value, so the software assign the default value of $0pF$ to all of them

Warning

Warning (306006): Found 103 output pins without output pin load capacitance assignment

4.2 Timing analyzer

To setup the timing analyzer is important to write the .sdc file where are defined the clock settings and the timing constrain for the input and output pins.

4.2.1 Timing configuration file

The clock signals in this project are three:

- **CLOCK_50**
is the 50 MHz clock from the oscillator on the DE2 board.
- **CLOCK_PLL**
is the clock output of the PLL. This clock source is generated from the **CLOCK_50** clock by multiplying it by 7/5.
This is a 70 MHz clock source with the same phase as **CLOCK_50**.
- **CLK_FREQ_DIV**
is the output of the frequency divider instantiated inside the **ddfs** module.
The problem with this clock source is that the frequency can be selected through the external pins of the module; for the analysis, it has been configured at its maximum frequency, which is **CLOCK_PLL** divided by two.

the part of code used to set up the clock signals is the following

```
1  create_clock
2      -name {CLOCK_50}
3      -period 20.000
4      -waveform {0.000 10.000}
5      { CLOCK_50 }
6
7  create_generated_clock
8      -name {CLOCK_PLL}
9      -source {inst_pll|altpll_component|pll|inclk[0]}
10     -divide_by 5 -multiply_by 7
11     -duty_cycle 50.00 -phase 0.0 -offset 0.0
12     { inst_pll|altpll_component|pll|clk[0] }
13
14  create_generated_clock
15      -name {CLK_FREQ_DIV}
16      -source {inst_pll|altpll_component|pll|clk[0]}
17      -divide_by 2 -multiply_by 1
18      -duty_cycle 50.00
19      { ddfs:inst_ddfs_mod|freq_divider_DDFS:f_div_inst|clk_out }
```

Listing 26: Clock settings for timing analyzer

Now that we have defined the clock, we must add the input and output timing constrain.
First we define the asynchronous input and output pins, in our case the asynchronous pins are the

- Switch input pins
- Seven segment display output pins
- led output pins
- GPIO_0 pins
this pins are connected the row of the keypad module, the pression of a button of the keypad can be asynchronous and the input is protected with a latch chain.

the part of code used to setup the asynchronous path is the following

```

1  set_false_path
2    -from [get_ports {GPIO_0[0] GPIO_0[1] GPIO_0[2] GPIO_0[3] SW[*]}]
3  set_false_path
4    -to [get_ports {HEX0[*] HEX1[*] ... HEX5[*] HEX6[*] HEX7[*]}]
5  set_false_path
6    -to [get_ports {LEDR[*] LEDG[*]}]

```

Listing 27: Setup asynchronous paths

Now it is time to set the timing constraints for the synchronized outputs.

To do this, we use the `set_input_delay` and `set_output_delay` commands.

These commands take two parameters: `-clock`, which is the reference clock for the specified input or output, and `-min` or `-max`, which refer to hold and setup timing constraints, respectively, and are referenced to the clock edge, as shown in Figure 31.

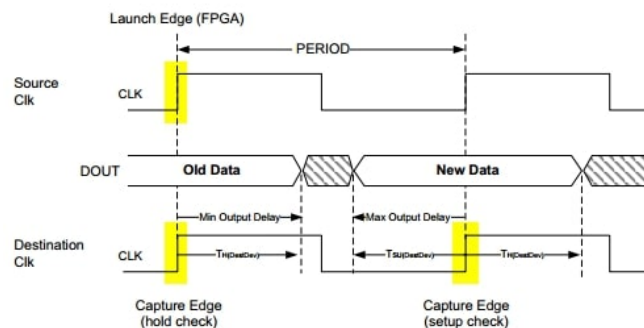


Figure 31: min and max delay timing interpretation

For the GPIO outputs, which are connected to the keypad, and for the LCD outputs, the signals are synchronized with the CLOCK_PLL source; however, the devices driven by these signals do not have a clock input. For the LCD, the timing constraints are controlled by the lcd_driver, and the keypad is a passive device, so it does not have digital circuits for synchronization. For this reason, the constraints for these pins are arbitrary. For the VGA output, we use the datasheet values for setup and hold times (respectively $1.5ns$ and $2.5ns$) and choose slightly larger values to ensure safe operation. The code to set up these timing constraints is the following

```

1      #the LCD and GPIO_1 outputs are not synchronized;
2      #therefore, the clock limits within a cycle are arbitrary
3      set_output_delay
4          -min 1 [get_ports {GPIO_1[0] GPIO_1[1] GPIO_1[2] GPIO_1[3]}]
5          -clock [get_clocks CLOCK_PLL]
6      set_output_delay
7          -max 1 [get_ports {GPIO_1[0] GPIO_1[1] GPIO_1[2] GPIO_1[3]}]
8          -clock [get_clocks CLOCK_PLL]
9      set_output_delay
10         -min 1 [get_ports LCD*]
11         -clock [get_clocks CLOCK_PLL]
12      set_output_delay
13         -max 1 [get_ports LCD*]
14         -clock [get_clocks CLOCK_PLL]
15
16      #from datasheet min = thold = 2.5ns; max = tsetup = 1.5ns
17      #we take some margin
18      set_output_delay
19         -min 3 [get_ports VGA*]
20         -clock [get_clocks CLK_FREQ_DIV]
21      set_output_delay
22         -max 2 [get_ports VGA*]
23         -clock [get_clocks CLK_FREQ_DIV]

```

Listing 28: Setup synchronous paths

4.2.2 Timing slack and critical path

The timing analyzer uses the library of the FPGA macro-cells to verify that the timing constraints are respected for both the slow and fast models.

The slow model is used to check the maximum frequency of the clock sources (at a temperature of 85°C) and the setup timing constraints.

The fast model is used to verify that hold time violations do not occur (at a temperature of 0°C).

Slow model

As we can see in Figure 32, the maximum calculated frequency for each clock source is higher than the frequency used.

	Fmax	Restricted Fmax	Clock Name	Note
1	62.42 MHz	62.42 MHz	CLK_FREQ_DIV	
2	79.1 MHz	79.1 MHz	CLOCK_PLL	

Figure 32: Maximum clock frequency for every clock source

The maximum frequency is a consequence of setup time violations; in fact, from the setup time slack, we can calculate the maximum frequency for each clock source.

The maximum frequency is only computed for paths where the source and destination registers or ports are driven by the same clock.

	Clock	Slack	End Point TNS
1	CLOCK_PLL	1.642	0.000
2	CLK_FREQ_DIV	3.873	0.000

Figure 33: Setup time slack

The slack reported in Figure 33 is related to the entire system, so the value represents the worst slack considering the interaction of the two clock sources.

The two paths that reduce the slack of the clock sources are shown in Figure 35 and in Figure 34

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	1.642	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.651
2	1.646	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.647
3	1.650	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.643
4	1.654	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.639
5	1.665	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.628
6	1.666	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.627
7	1.668	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.625
8	1.670	keypad_freq_sel_mod..._inst final_freq[0]	keypad_freq_sel_mod...st mult_1000000[42]	CLOCK_PLL	CLOCK_PLL	14.285	0.055	12.623

Figure 34: CLOCK_PLL setup time slack

	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	3.873	inst_dffe_4	VGA_R[9]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.668
2	3.913	inst_dffe_4	VGA_R[9]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.628
3	3.942	inst_dffe_4	VGA_R[9]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.599
4	4.112	inst_dffe_4	VGA_R[9]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.429
5	4.170	inst_dffe_4	VGA_R[6]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.371
6	4.191	inst_dffe_4	VGA_R[8]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.350
7	4.210	inst_dffe_4	VGA_R[6]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.331
8	4.231	inst_dffe_4	VGA_R[8]	CLOCK_PLL	CLK_FREQ_DIV	14.286	0.255	8.310

Figure 35: CLK_FREQ_DIV setup time slack

Critical path

From the analysis done with the slow model for the setup time slack, we can see that for the CLOCK_PLL source, the critical path is referred to the shift and add operation done into the DDFS_frequency_converter module

```
1      mult_1000000 <=
2      (final_freq<<19)+(final_freq<<18)+(final_freq<<17)+(final_freq<<16)
3      +(final_freq<<14)+(final_freq<<9)+(final_freq<<6)
```

For the CLK_FREQ_DIV source, indeed the critical path came from the connection between the mirror input of the DDFS module and the DAC output.

The critical path is referred to this two combinational blocks inside the module

```
1      if(mirror_x && mirror_y) begin
2          //third quarter or fourth quarter
3          ...
4
5      end else if(mirror_x) begin
6          //second quarter or fourth quarter
7          ...
8
9      end
```

```
1      if(mirror_y)
2          //second quarter or fourth quarter
3          ...
```

This blocks was previously analyzed in section 2 (Listing 3 and Listing 2).

In the Chip Planner, we can see where these two critical paths are located inside the FPGA.

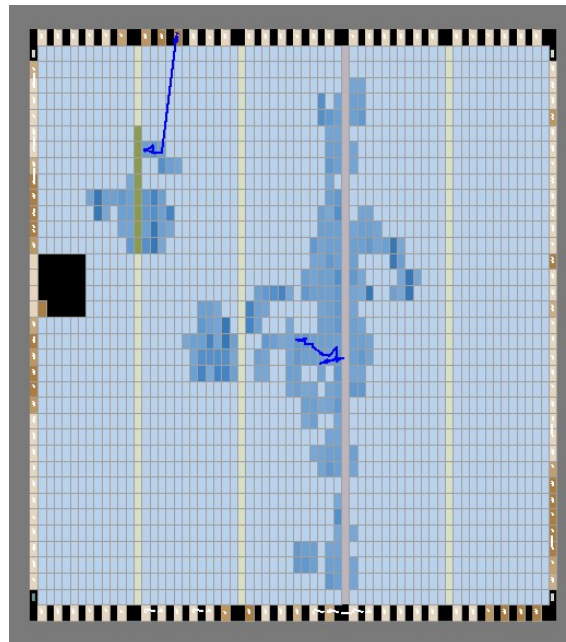


Figure 36: Critical Path shown in the Chip Planner

Fast model

The hold time slack for every clock source is shown in Figure 37

	Clock	Slack	End Point TNS
1	CLOCK_PLL	0.215	0.000
2	CLK_FREQ_DIV	0.469	0.000

Figure 37: Hold time slack

To obtain this values of hold time slack we optimized all the path to increase the hold time slack. To do this, we modified the Fitter settings as shown in Figure 38



Figure 38: Fitter settings

4.3 Power consumption

At the end of the Analysis and Synthesis, we run the Power Analyzer to check the power consumption of the project.

This analysis does not consider the power dissipated to drive the loads, because by default, the load capacitances are set to 0 pF.

To perform a more accurate analysis, we have to estimate the load capacitance for each output of the circuit.

PowerPlay Power Analyzer Summary	
PowerPlay Power Analyzer Status	Successful - Sat Jan 03 20:17:01 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	main_function_generator
Top-level Entity Name	main_function_generator
Family	Cyclone II
Device	EP2C35F672C6
Power Models	Final
Total Thermal Power Dissipation	229.34 mW
Core Dynamic Thermal Power Dissipation	77.92 mW
Core Static Thermal Power Dissipation	80.32 mW
I/O Thermal Power Dissipation	71.10 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

Figure 39: Power analyzer summary

The analyzer tool also allows you to see which parts of the power are dissipated and which portions are consumed by each block, as shown in Figure 40

	Block Type	Total Thermal Power by Block Type	Block Thermal Dynamic Power	Thermal Static Pow	Routing Thermal Dynamic Power	Block Average Toggle Rate (millions of transtions / sec)
1	PLL	7.48 mW	7.48 mW	--	0.00 mW	140.007
2	I/O	41.91 mW	28.44 mW	13.08 mW	0.38 mW	6.214
3	M4K	5.14 mW	5.12 mW	--	0.02 mW	3.416
4	Embedded multiplier block	6.74 mW	6.74 mW	--	0.00 mW	30.158
5	Embedded multiplier output	14.51 mW	0.00 mW	--	14.51 mW	34.746
6	Combinational cell	21.61 mW	12.81 mW	--	8.81 mW	14.246
7	Register cell	10.66 mW	5.53 mW	--	5.13 mW	8.672
8	Clock control block	11.38 mW	0.00 mW	--	11.38 mW	105.004

Figure 40: Termal power dissipation by block type

4.4 Resource usage

The internal FPGA resources used at the end of the Analysis and Synthesis operations are reported in Figure 41.

	Resource	Usage
1	Estimated Total logic elements	2,830
2		
3	Total combinational functions	2791
4	▼ Logic element usage by number of LUT inputs	
1	-- 4 input functions	507
2	-- 3 input functions	1519
3	-- <=2 input functions	765
5		
6	▼ Logic elements by mode	
1	-- normal mode	948
2	-- arithmetic mode	1843
7		
8	▼ Total registers	789
1	-- Dedicated logic registers	789
2	-- I/O registers	0
9		
10	I/O pins	109
11	Total memory bits	29696
12	Embedded Multiplier 9-bit elements	70
13	▼ Total PLLs	1
1	-- PLLs	1
14		
15	Maximum fan-out node	ALTPLLa:inst_pll altpll:altpll_component _clk0
16	Maximum fan-out	784
17	Total fan-out	11364
18	Average fan-out	3.01

Figure 41: Resource usage table

Compared to all the resources available on the chip, the total logic elements used are only 9% of the total, and the total memory bits used are 6%, as shown in Figure 42.

Furthermore has been used all the embedded multiplier and one of the four internal PLL.

Flow Summary	
Flow Status	Successful - Sat Jan 03 20:17:01 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	main_function_generator
Top-level Entity Name	main_function_generator
Family	Cyclone II
Device	EP2C35F672C6
Timing Models	Final
Total logic elements	3,011 / 33,216 (9 %)
Total combinational functions	2,967 / 33,216 (9 %)
Dedicated logic registers	700 / 33,216 (2 %)
Total registers	700
Total pins	109 / 475 (23 %)
Total virtual pins	0
Total memory bits	29,696 / 483,840 (6 %)
Embedded Multiplier 9-bit elements	70 / 70 (100 %)
Total PLLs	1 / 4 (25 %)

Figure 42: Analysis and synthesis flow summary

In the Chip Planner, we can see where are placed all the blocks of the project, inside the FPGA, as shown in Figure 43

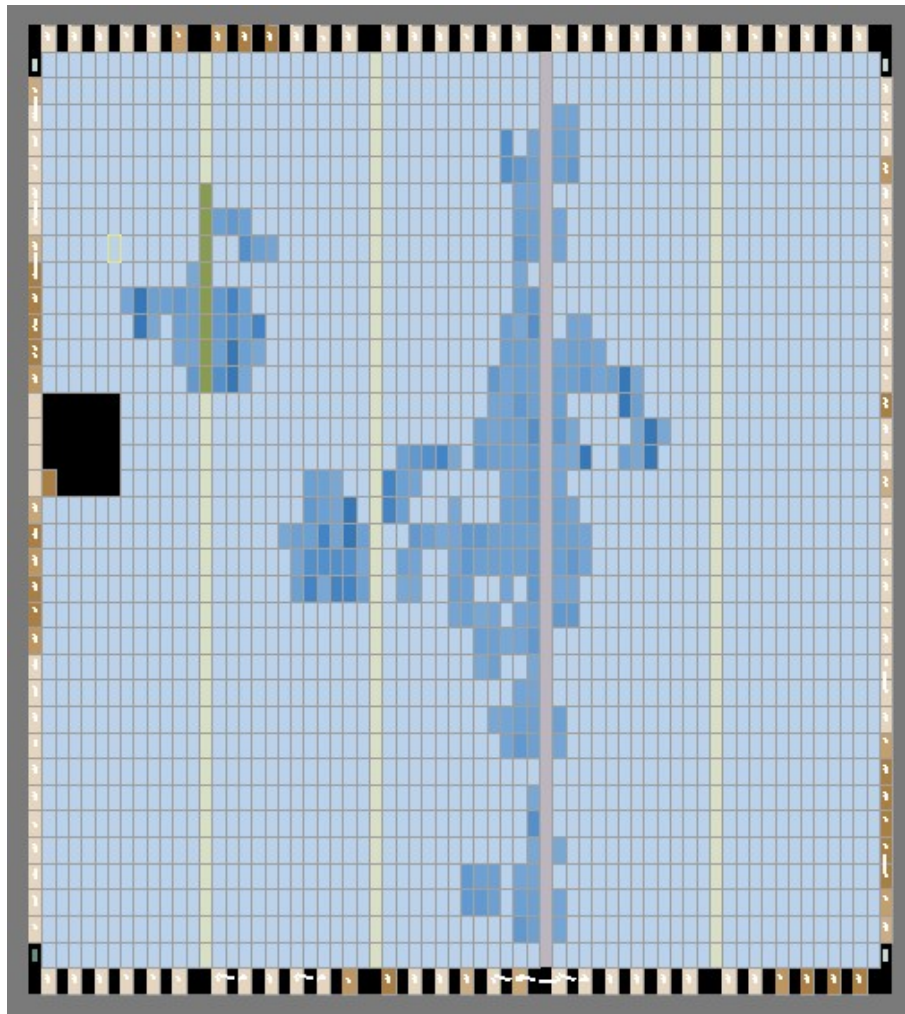


Figure 43: Chip Planner

5 Conclusions

We recall that the goal of the project was to design a function generator capable of producing output waveforms with frequencies up to 1 MHz .

Laboratory tests have shown that the proposed design fully meets the specified requirements.

Nevertheless, some parts of the system could be optimized, and several possible upgrades may be implemented to enhance the overall functionality of the design without significantly increasing the hardware complexity of the chip.

5.1 Criticality

Even though it has been verified with the timing analyzer that the dynamic clock generated by the frequency divider instantiated in the DDFS module does not cause any timing violations, this verification has been carried out by considering the clock source in phase with the main clock, without taking into account the t_{co} of the flip-flop.

To avoid potential issues, the divided clock can be considered asynchronous and the input can be protected using a flip-flop chain.

A more professional solution can be obtained by using a `clock_enable` signal, which identifies, via a counter synchronized with the main clock, the n -th clock cycle and synchronizes the module using this signal, where n is controlled by the `freq_control` input.

A possible Verilog implementation of the clock enable signal is shown in Listing 29.

```
1  integer cont;
2  always @(posedge clk) begin
3      if(rst_n == 0) begin
4          //reset condition
5          cont <= 0;
6          clk_ena <= 1'b0;
7      end
8      else begin
9          if(cont >= n) begin
10             cont <= 0;
11             clk_ena <= 1'b1;
12         end
13         else begin
14             cont <= cont + 1;
15             clk_ena <= 1'b0;
16         end
17     end
18 end
```

Listing 29: Implementation of clock enable signal with frequency divider

With the `clock_ena` signal, all the sequential blocks synchronized with the divided frequency clock become like the block in Listing 30

```
1  always @(posedge clk) begin
2      if(clk_ena == 1) begin
3          //divided clock operation
4      end
5      else begin
6          //store the signals
7      end
8  end
```

Listing 30: Avoid using divided clock

5.2 Possible optimizations

The first hardware optimization that can be done is inside the DDFS_frequency_converter where a large number of multipliers have been instantiated.

```
1  always @(*) begin
2
3      if(final_freq <= (CLK_FREQ/(1000000*MIN_LUT_SAMPLE))) begin
4          freq_control = 3'd6;
5          fw = (fw_1000000 >= 0) ? fw_1000000 : 7'd0;
6      end
7      else if(final_freq <= (CLK_FREQ/(100000*MIN_LUT_SAMPLE))) begin
8          freq_control = 3'd5;
9          fw = (fw_100000 >= 0) ? fw_100000 : 7'd0;
10     end
11     .
12     .
13     .
14
15     else begin
16         //default condition, max frequency
17         freq_control = 3'd0;
18         fw = 7'd0;
19     end
20
21 end
22
23 always @(posedge clk) begin
24     ...
25
26     fw_1000000 <= (mult_1000000*K)>>54;
27     fw_100000 <= (mult_100000*K)>>54;
28     fw_10000 <= (mult_10000*K)>>54;
29     fw_1000 <= (mult_1000*K)>>54;
30     fw_100 <= (mult_100*K)>>54;
31     fw_10 <= (mult_10*K)>>54;
32     fw_2 <= (mult_2*K)>>54;
33 end
```

Listing 31: Part of the calculations performed to obtain the frequency word value

In Listing 31, is calculated the value of f_w for every clock value that goes into the DDFS module. fw_2 is the frequency word calculated with the clock divided by two, fw_{10} is the same with the clock divided by ten and so on.

The problem is that `mult_*` is a *64bit* value, so the multipliers instantiated are big combinatorial network. To reduce the number of multiplier instantiated the input of the operation can be multiplexed as shown in Listing 32

```

1      always (*) begin
2
3          if(final_freq <= (CLK_FREQ/(1000000*MIN_LUT_SAMPLE))) begin
4              freq_control = 3'd6;
5              mult = mult_1000000;
6          end
7          else if(final_freq <= (CLK_FREQ/(100000*MIN_LUT_SAMPLE))) begin
8              freq_control = 3'd5;
9              mult = mult_100000;
10         end
11         .
12         .
13         .
14
15         else begin
16             //default condition, max frequency
17             freq_control = 3'd0;
18             mult = mult_2;
19         end
20
21     end
22
23     always (posedge clk) begin
24         ...
25
26         fw <= (mult*K)>>54;
27     end

```

Listing 32: Example to optimize the calculation of frequency word value

This optimization instantiates a multiplexer and can reduce the maximum frequency of the project; to overcome this problem, the multiplexing operation can be pipelined by modifying the combinatorial block into a synchronous block.

The pipeline operation increases the frequency but also the throughput of the system.

Another optimization that can be done is to allow the `ddfs` module to work with its own clock source and control it asynchronously using a FIFO memory.

This is because the `ddfs` module can work at a frequency much higher than the other modules, due to its hardware simplicity, and the other modules are used to interface with the operator, so they can produce the outputs even after some milliseconds.

This upgrade allows the frequency generator to produce higher-frequency waveforms without increasing the complexity of the circuit.

Another possible optimization concerns the LUTs that store the contents of the screens displayed on the LCD. For future upgrades, some spaces are left blank to allow the addition of more pages. However, in the final version, all unused blank spaces can be removed, reducing the amount of instantiated LUT memory.

5.3 Possible upgrades

A simple upgrade that can be made is to instantiate two additional `ddfs` modules within the project and assign each module a dedicated button on the keypad.

By doing this, we can use all three DAC outputs to generate dedicated waveforms.

A more advanced update could also allow the modules to communicate with each other, enabling their outputs to be synchronized, either in phase or with a programmed phase shift.

Furthermore, it is possible to implement a method, via USB communication or SD card, to upload samples of customizable waveforms for display. These samples are then saved into the board's internal FLASH memory and can be displayed when needed.

Before displaying a custom waveform, it is important to transfer the samples from FLASH to internal RAM, since the read latency of FLASH significantly reduces the output frequency of the waveform.